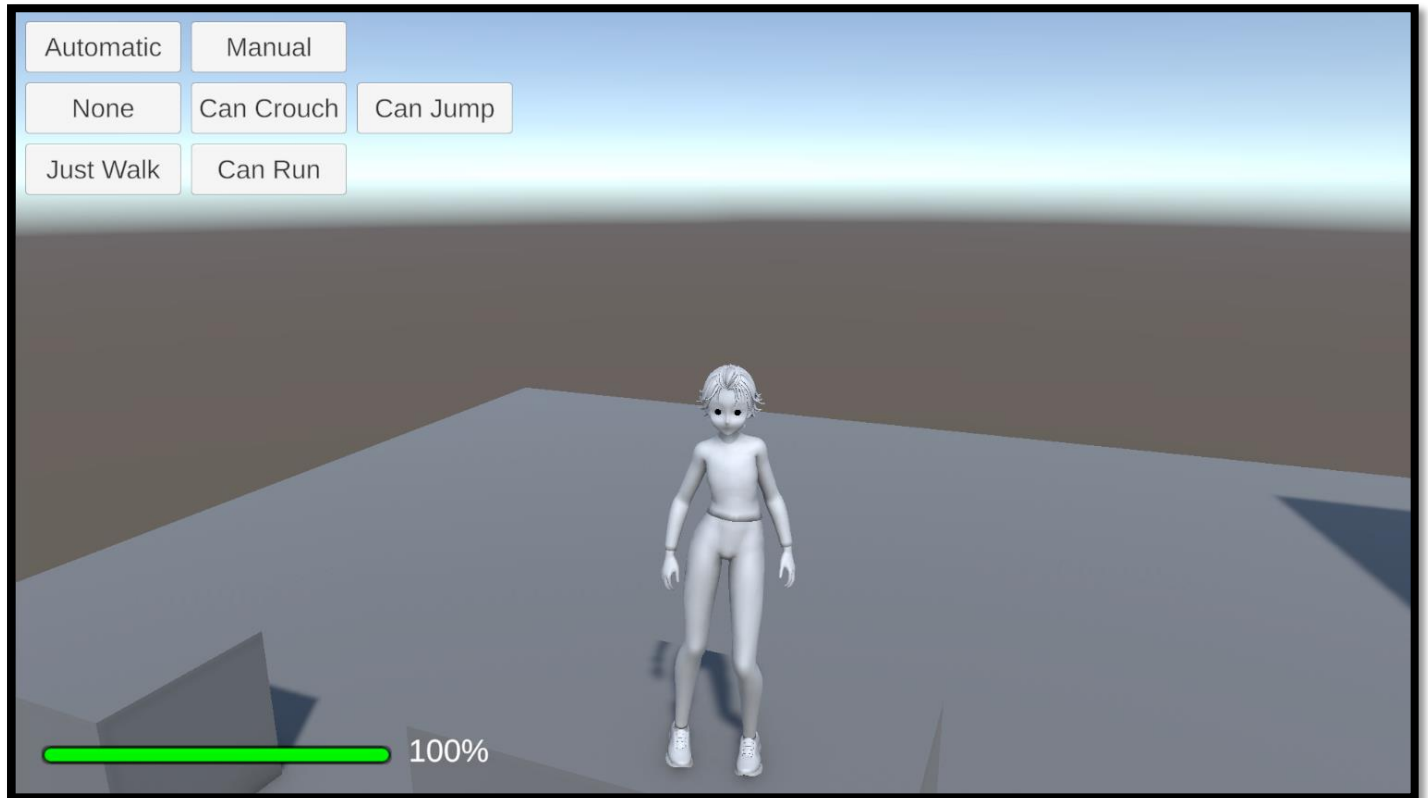




Player Controller

Documentation

Creator: Lucas Gomes Cecchini

Online Name: AGAMENOM

Overview

This document will help you to use **Player Controller** for **Unity**.

With it, you can create a gameplay for your game with a modular system that's compatible with Unity's **Input System**.

It also comes with some **Predefined Controllers**.

Instructions

You can get more information from the Playlist on YouTube:

<https://www.youtube.com/playlist?list=PL5hnfx09yM4LxhQjXoLIwwwmEuMW9LySa>

Script Explanations

Box Collision Sensor

Purpose Overview

The BoxCollisionSensor class provides **runtime box-shaped collision detection** using Unity's physics system (Physics.OverlapBoxNonAlloc).

It allows detection control, collision filtering, and debug visualization—all dynamically configurable through external providers (functions passed into the constructor).

This makes it ideal for advanced systems like **modular player controllers**, **AI sensing**, or **environment interaction**, where detection parameters change during gameplay.

Providers (Runtime Configuration Inputs)

These are function delegates (Func<T>) that return dynamic data every frame.

Instead of storing static values, the sensor calls these functions when needed, making the system flexible and reactive to runtime changes.

Provider Variable	Type	Description
enableDetectionProvider	Func<bool>	Controls if collision detection should run. If it returns false, detection is skipped.
enableDebugLogProvider	Func<bool>	Enables console logging when a collision is detected.
filterModeProvider	Func<DetectionFilter>	Decides what filtering rule is applied to detected colliders (see DetectionFilter enum).
referenceParentTransformProvider	Func<Transform>	Used for filtering — excludes colliders that are children of a specific transform.
ignoredTagsProvider	Func<HashSet<string>>	Contains tags that should be ignored in detection (e.g., "Player", "Ally").
collisionLayerMaskProvider	Func<LayerMask>	Determines which Unity physics layers are considered for collisions.
triggerInteractionProvider	Func<QueryTriggerInteraction>	Decides if triggers are included in collision detection.

boxCenterProvider	Func<Vector3>	Provides the world-space position of the box's center.
boxSizeProvider	Func<Vector3>	Provides the dimensions of the detection box (width, height, depth).
boxRotationProvider	Func<Quaternion>	Provides the rotation of the box in world space.
gizmoTargetObjectProvider	Func<GameObject>	Determines which GameObject is used as a gizmo reference (useful in editor mode).
gizmoDrawingModeProvider	Func<GizmoDisplayMode>	Decides when gizmos should be drawn (Always, SelectedOnly, None).
gizmosColorProvider	Func<Color>	Provides the color used when drawing the box gizmo.

Enums (Internal Configuration)

DetectionFilter

Controls how detected colliders are filtered:

- **None:** No filtering, all colliders are valid.
- **IsNotChildOf:** Ignores colliders that are children of a given parent transform.
- **IgnoreByTags:** Ignores colliders whose tags appear in a list of ignored tags.
- **All:** Combines both the above filters.

GizmoDisplayMode

Determines when gizmos should appear in the Unity editor:

- **None:** Never draw.
- **Always:** Always draw.
- **SelectedOnly:** Only draw when the object is selected.

Fields and Runtime Data

Field	Description	Field
hitResults	A reusable array (size 16) for storing colliders detected by Physics.OverlapBoxNonAlloc. This avoids memory allocations each frame.	hitResults

collisionDetected

A public boolean flag that indicates whether a collision was found during the last detection cycle.

collisionDetected

Constructor

When the sensor is created, it:

1. Stores all provider functions (using defaults when not provided).
2. Registers itself with the **PhysicsUpdateBroadcaster**, subscribing to its **OnFixedUpdate** event.
 - This ensures CheckForCollisions() runs automatically every physics step.
3. Registers itself internally (PhysicsUpdateBroadcaster.AddSensor(this)), likely for debugging or editor visualization.

This design allows the sensor to “live” as long as it is registered, updating automatically each frame.

Dispose Method

A static helper that safely cleans up a sensor:

1. Unsubscribes it from OnFixedUpdate.
2. Removes it from the broadcaster’s list.
3. Nullifies the reference (to avoid memory leaks or dangling pointers).

Example of use:

```
BoxCollisionSensor.Dispose(ref mySensor);
```

Core Logic

CheckForCollisions

This private method is automatically called during each **FixedUpdate** via the broadcaster.

It performs the main detection:

1. Checks if detection is enabled (enableDetectionProvider()).
 - If not, exits early.
2. Resets the collisionDetected flag.
3. Retrieves all necessary parameters (center, size, rotation, layer mask, etc.) via the providers.
4. Calls Unity’s **Physics.OverlapBoxNonAlloc**, which checks for colliders overlapping a box volume:
 - The box’s **center** and **half-size** define its shape.
 - **Rotation**, **LayerMask**, and **TriggerInteraction** control how the query behaves.
 - Results are written into the hitResults array without new memory allocations.

5. Iterates through all detected colliders.
 - For each, it runs `PassesDetectionFilter()` to check if it's valid.
 - If valid, optionally logs to the console (`enableDebugLogProvider()`).
 - Sets `collisionDetected = true` and stops checking further (since one valid hit is enough).

PassesDetectionFilter

Applies the chosen filtering rules to each detected collider.

Steps:

1. Gets the current filter type.
2. Gets the reference parent and ignored tags from their providers.
3. Based on the filter type, applies the exclusion logic:
 - **None**: Always true.
 - **IsNotChildOf**: Exclude if collider belongs to a child of the reference transform.
 - **IgnoreByTags**: Exclude if collider's tag is in ignored list.
 - **All**: Combines both checks (must pass both).
4. Returns true if the collider passes all tests.

Usage Example

Here's how you could **use** this system conceptually (not in C# code syntax):

1. **Define providers** that supply the current position, size, etc. — often taken from a player's transform or dynamic system.
2. **Create a BoxCollisionSensor** using these providers.
3. The sensor automatically performs detection every physics update.
4. You can read the `collisionDetected` flag to know if a collision occurred.
5. When done, call `Dispose()` to cleanly remove it from the update cycle.

Summary of Behavior

- Detects collisions in a **box-shaped area** each physics step.
- Fully configurable at runtime through **provider functions**.
- Can log detected collisions and visualize the detection area in the **Unity Editor**.
- Integrates with a **PhysicsUpdateBroadcaster** for automatic update handling.
- Supports **filtering by hierarchy and tags, layer masks, and trigger inclusion**.
- Optimized with **no allocations** per frame and **safe disposal**.

Check Valid Angle Sensors

Overview

This system performs a **downward raycast** from the player's position every physics frame. It determines the slope angle of the surface below the player, decides if it's within an acceptable range, and changes the **physics friction** of the player's colliders accordingly.

It's especially useful in **character controllers** to detect when the player is grounded, slipping on a steep slope, or walking on stable terrain.

Providers (Dynamic Data Sources)

Each provider is a **function reference** that supplies real-time data to the system. Instead of fixed values, these are small "getters" that can change every frame, making the system highly adaptable.

Provider	Purpose
targetTransform	Returns the player's position and orientation. Used as the raycast origin.
maxStableAngle	Returns the maximum slope angle that counts as "walkable." (Default is 45° if not provided.)
raycastDistance	Returns how far downward the raycast should check for ground. (Default is 0.5 units.)
highFrictionMaterial	Returns the material applied when the player stands on a stable, walkable slope.
lowFrictionMaterial	Returns the material applied when the player stands on an unstable or steep slope.
colliders	Returns the list of player colliders that will have their material changed depending on the slope.
debug	Returns a boolean that controls whether to draw debug visuals and print logs in the console.

All these values are supplied externally when the system is created, allowing flexible configuration from other scripts or gameplay systems.

Internal Fields

Field	Description	Field
isValidAngle	A boolean flag that stores whether the last detected slope was valid (true for walkable ground, false for too steep or no ground).	isValidAngle
raycastHits	A reusable array used to store results of the raycast. It avoids creating new memory objects each frame, improving performance.	raycastHits

Initialization

When an instance of this system is created:

1. All provider functions are stored for later use.
2. Default values are assigned for optional providers (45° slope, 0.5 distance, and debug disabled).
3. The system subscribes to the **PhysicsUpdateBroadcaster**, which calls its main detection method (**CheckRaycast**) automatically on every **physics update**.

This means that once initialized, the system runs automatically at each physics step without needing manual calls.

Disposal

A static utility method allows safe removal of the system:

- It unsubscribes from the physics update event.
- It clears its own reference to prevent lingering memory usage or multiple instances running simultaneously.

This is important when disabling a character, unloading a scene, or cleaning up after a temporary object.

Core Detection Logic (**CheckRaycast**)

The heart of the system is the **CheckRaycast** method.
It runs every physics frame and performs the following steps:

1. Setup

- Retrieves the player's position from `targetTransform`.
- Moves the ray origin slightly upward to avoid hitting the player's own colliders.
- Gets the detection distance and maximum slope angle from their providers.

2. Perform Downward Raycast

- Casts a **ray downward** from the player's position up to the defined distance.
- Uses a **non-allocating raycast**, meaning it reuses the same results array instead of creating new ones — an optimization for real-time performance.
- If no hit is found, it marks `isValidAngle` as false and optionally logs a red debug line showing "no ground detected."

3. Process Hits

- If one or more surfaces were hit, the hits are sorted by **distance**, ensuring the nearest surface is processed first.

- The system iterates over each hit in order:
 - **Skips** any collider that belongs to the player itself.
 - **Ignores** surfaces that are too vertical (e.g., walls), determined by checking if the upward component of the surface normal is too small ($\text{normal.y} < 0.5$).

4. Calculate Slope Angle

- Calculates the **angle** between the surface normal and the world's upward direction.
- Compares this angle to the **maximum stable angle**.
 - If below or equal to the limit, the slope is **stable** (walkable).
 - If greater, the slope is **unstable** (too steep).

5. Apply Friction Material

- Selects the appropriate **physics material**:
 - Stable → **high friction** (the player can walk and stop easily).
 - Unstable → **low friction** (the player slides or loses grip).
- Applies the chosen material to all player colliders provided by the colliders provider.
- Updates `isValidAngle` to reflect whether the surface is walkable.

6. Debug Visualization

If debug mode is enabled:

- Draws colored lines in the scene view to show the ray and the surface normal.
 - **Green** for stable slopes.
 - **Yellow** for unstable slopes.
 - **Red** when no valid ground was found.
- Logs information or warnings in the console showing:
 - Surface name.
 - Detected angle.
 - Whether it's stable or not.

7. Exit Conditions

- The system exits after handling the first valid hit, meaning it only reacts to the **closest valid surface** beneath the player.
- If all hits are ignored (for example, because they were part of the player or too steep), it marks the result as invalid and optionally logs that no valid ground was detected.

The class includes an **internal helper** that sorts raycast results by distance. This ensures the system processes the nearest hit first, which is crucial for accurate grounding detection — especially if there are multiple overlapping colliders below the player.

Usage Scenario

1. A gameplay system (like a player controller) creates an instance of this detector, providing the required functions (for example, how to get the player's position, colliders, and friction materials).
2. Once active, the detector automatically monitors the ground each physics frame.
3. The controlling system can check the `isValidAngle` flag to decide:
 - Whether the player is grounded.
 - Whether the player can walk or should slide.
4. The player's colliders automatically switch materials to simulate sliding or traction depending on slope angle.
5. When no longer needed, the system is disposed to stop updates.

Behavior Summary

- Casts a downward ray every physics step.
- Checks whether the ground is within a safe slope angle.
- Updates the friction of player colliders dynamically.
- Displays optional debug lines and logs.
- Runs efficiently without creating new objects each frame.
- Automatically integrates into Unity's physics timing via a broadcaster.

Custom Player Data

Overview

This script defines a helper class designed to **serialize** and **deserialize** custom player data in a Unity project.

Its purpose is to allow you to store a dictionary of player-related values (like position, health, level, etc.) into a **JSON string**, and later restore it back into usable data types.

It supports:

- Normal C# primitive types (int, float, bool, string).
- Unity types (like Vector2, Vector3, Quaternion).
- Enums.
- Any custom types with proper serialization support.

Everything is stored as key–value pairs with **type metadata**, meaning the script remembers what type each value was when it was saved.

Namespace

namespace PlayerController.CustomData

This groups the class logically within the game's codebase — indicating it belongs to the Player Controller system and is related to custom data management.

Class: CustomPlayerData

This is the main class.

It contains all logic for saving (serializing) and loading (deserializing) player data.

The class is **static** in practice (all public methods are static), which means you don't instantiate it — you simply call its methods directly.

Example usage:

```
CustomPlayerData.SaveData(...)
```

```
CustomPlayerData.LoadData(...)
```

Inner Classes

1. Entry

This private class represents a single key-value pair that will be stored in JSON form.

It has three public fields:

- **key** — The name or identifier for the piece of data (e.g., "PlayerHealth").
- **value** — The serialized value as a string (e.g., "100" or "{x:1, y:2}" for Unity types).
- **type** — The fully qualified type name (e.g., "System.Int32" or "UnityEngine.Vector3").

Each Entry object corresponds to one item in the saved data dictionary.

2. EntryList

A wrapper class that contains a list of Entry objects.

It has:

- **entries** — A List<Entry> that stores all serialized key-value pairs together.

This is needed because JsonUtility (Unity's JSON serializer) cannot directly serialize a dictionary, but it can serialize classes and lists.

Serialization Section

Method: SaveData

Purpose:

Converts a dictionary of data (provided by a delegate) into a JSON string for saving.

Parameters:

- **dataBuilder** — A function (Func<Dictionary<string, object>>) that returns the dictionary of data you want to save.
This allows you to define how to build the player's data dynamically.

Returns:

A string containing the serialized JSON data.

How it works step by step:

1. It calls dataBuilder() to get the player data dictionary.
2. It creates an empty EntryList to hold serialized entries.
3. For each key-value pair in the dictionary:
 - It skips null values.
 - It checks the **type** of the value.
 - If it's an **enum**, it stores the enum name as a string.
 - If it's a **Unity type** (detected by checking the namespace "UnityEngine"), it uses JsonUtility.ToJson() to properly serialize it.
 - Otherwise, it calls .ToString() as a fallback.
4. For each serialized item, it creates a new Entry with:
 - The key.
 - The serialized value.
 - The full type name (via AssemblyQualifiedName so it can be reconstructed later).
5. It adds each entry to the list.
6. Finally, it serializes the entire EntryList to a JSON string using JsonUtility.ToJson() and returns it.

Usage Example:

```
string json = CustomPlayerData.SaveData(() => new Dictionary<string, object>
{
    { "Health", 100 },
    { "Position", player.transform.position },
```

```
{ "IsAlive", true },  
  { "State", PlayerState.Running }  
});
```

This will generate a JSON string that represents the player's health, position, state, etc.

Deserialization Section

Method: LoadData

Purpose:

Takes a previously saved JSON string, reconstructs the dictionary, and then gives it back via a callback function.

Parameters:

- **json** — The JSON string produced by SaveData.
- **applyData** — An action (Action<Dictionary<string, object>>) that receives the reconstructed data so it can be applied to the player.

How it works step by step:

1. It deserializes the JSON back into an EntryList using `JsonUtility.FromJson<EntryList>(json)`.
2. It creates an empty dictionary to store the reconstructed data.
3. It defines a set of **type converters** — small functions that know how to convert a string back into a specific type:
 - float, bool, int, string
 - Unity types (Vector2, Vector3, Vector4, Quaternion)
4. For each Entry in the list:
 - It looks up the original type name using `Type.GetType(entry.type)`.
 - If the type is missing, it logs a warning and skips that entry.
 - If the type is an **enum**, it parses the string back to its enum value.
 - If there's a known converter for that type, it uses it.
 - Otherwise, it attempts to convert using `Convert.ChangeType()`.
5. Each reconstructed value is added to the dictionary under its original key.
6. Finally, it calls `applyData(dict)` — letting you apply the restored values to your game objects or player state.

Usage Example:

```
CustomPlayerData.LoadData(json, data =>  
{
```

```

player.Health = (int)data["Health"];

player.transform.position = (Vector3)data["Position"];

player.IsAlive = (bool)data["IsAlive"];

player.State = (PlayerState)data["State"];

});

```

Summary of Key Points

Component	Purpose
Entry	Holds a single key, value, and type info.
EntryList	Wraps a list of entries for serialization.
SaveData()	Converts a data dictionary into JSON.
LoadData()	Converts a JSON string back into a data dictionary.
dataBuilder	Function that generates the data to be saved.
applyData	Function that applies the loaded data back into the game.
converters	Provides specific parsing logic for Unity types and primitives.

Practical Usage Scenario

Imagine you want to save your player's progress when leaving the game and load it later:

Saving Example:

```

string saveJson = CustomPlayerData.SaveData(() => new Dictionary<string, object>
{
    { "Health", player.Health },
    { "Position", player.transform.position },
    { "Rotation", player.transform.rotation },
    { "IsAlive", player.IsAlive },
    { "State", player.CurrentState }
});

```

```
// Then write `saveJson` to a file or PlayerPrefs.
```

Loading Example:

```
CustomPlayerData.LoadData(savedJson, data =>
{
    player.Health = (int)data["Health"];
    player.transform.position = (Vector3)data["Position"];
    player.transform.rotation = (Quaternion)data["Rotation"];
    player.IsAlive = (bool)data["IsAlive"];
    player.CurrentState = (PlayerState)data["State"];
});
```

Mono Behaviour Extensions

Overview

This script provides a system that allows any Unity behaviour component to be inspected and interacted with dynamically during runtime, even if the class type is unknown in advance. It uses a technique called **reflection**, which allows the program to look inside other objects, discover their properties, and read their values.

The goal is to make it possible to detect and use **movement-related states** (like “is grounded”, “is running”, etc.) from any controller script — without directly depending on its implementation.

General Structure

The script is divided into two main regions:

1. **MonoBehaviour Extensions** — provides tools to read property values dynamically.
2. **Movement State Reflection Helpers** — defines a structure and adapter to extract movement information using the tools above.

Region 1 – MonoBehaviour Extensions

Purpose

This section defines a helper system that adds reflection-based capabilities to any Unity behaviour component.

It allows one to request the value of a public property by name, without having to know its exact class or interface.

Static Extension Class

A static utility container extends Unity's base behaviour class with an additional ability to "try and get" a property dynamically.

This is achieved through reflection, meaning it can look up public information about an object at runtime.

Method: TryGetProperty

Functionality

This method is designed to retrieve the value of a public property from a component by specifying its name as text.

It performs type checking to ensure that the value returned matches the expected data type.

Parameters Explained

- **mono** – The component being inspected. This is the target object whose properties will be accessed.
- **propName** – The name of the property you want to read (for example, "IsGrounded").
- **value** – A variable that will hold the property's value if the lookup succeeds.

Internal Process

1. The method inspects the target object's structure to locate a public property with the given name.
2. If such a property exists and its type matches the requested type, its value is read and returned.
3. If no matching property is found, or if the type does not match, a default (empty) value is returned instead.
4. The method indicates whether the lookup was successful or not.

Practical Effect

This allows developers to write generalized code that can interact with different controller scripts — even if their classes were written by other developers — as long as they expose the expected public properties.

Region 2 – Movement State Reflection Helpers

This region introduces an **interface** and an **adapter class** that together allow a game system to query movement-related states (grounded, moving, crouching, running) from any compatible component, using the reflection system defined above.

Interface: Player Movement State

Purpose

This interface defines a set of four boolean properties that represent the key movement conditions of a player or character.

It acts as a contract: any object that claims to represent a movement state must provide these four properties:

1. **IsGrounded** – True if the player is currently touching the ground.
2. **IsMoving** – True if the player is currently walking or otherwise changing position.
3. **IsCrouching** – True if the player is currently in a crouched stance.
4. **IsRunning** – True if the player is currently running.

The interface itself doesn't contain any logic; it simply declares what information is expected to exist.

Class: Player Movement State Adapter

Purpose

This adapter provides a bridge between any Unity component and the movement state interface described above.

It allows you to wrap any component — even if it doesn't officially implement the interface — and still read its public movement-related properties.

Internal Variable

- **target** – A reference to the component being inspected. This is the object from which property values will be read.

Constructor

When you create an adapter, you pass in the component you want to inspect. The adapter stores this component internally so it can later look up its properties.

Properties of the Adapter

Each property in the adapter corresponds to one of the expected movement states. They all use the same logic pattern: attempt to retrieve the named property from the target component using reflection, and if successful, return its value.

1. **IsGrounded** – Checks whether the target component has a property named “IsGrounded”. If found and true, this property will return true.
2. **IsMoving** – Works the same way, returning true if the target’s “IsMoving” property exists and is true.
3. **IsCrouching** – Reflectively reads “IsCrouching” and returns its value.
4. **IsRunning** – Reflectively reads “IsRunning” and returns its value.

If any of these properties are missing or not valid, the adapter simply returns false for that condition.

How It All Works Together

1. **At runtime**, you have a component (for example, a player controller) that exposes public properties like *IsGrounded*, *IsMoving*, *IsRunning*, etc.
2. You create a new instance of the **adapter**, passing in that component.
3. The adapter will now be able to query those properties dynamically using the **TryGetProperty** method from the extensions section.
4. Any system that requires the **movement state** can then use the adapter through the **movement interface**, ensuring consistency and flexibility.

Practical Usage Example (Conceptual)

1. Suppose you have two different player controllers made by different programmers.
 - One has properties named *IsGrounded* and *IsRunning*.
 - The other also has those properties, but belongs to a completely different class.
2. You can still use the same adapter to extract their state, without changing their code.
3. You simply pass the controller to the adapter, and the adapter will reflectively read those properties.
4. Your game systems (for example, an animation controller or HUD display) can then read from the adapter using the **movement interface**, without knowing which specific controller class is being used.

Advantages

- **Flexibility** – Works with any component that has the expected property names, regardless of its class type.
- **No hard dependencies** – You don’t need to directly reference or modify other scripts.
- **Runtime discovery** – Automatically adapts to components that expose the right public members.
- **Safety** – The reflection lookup includes validation to prevent type mismatches or missing properties from causing errors.

Summary Table

Element	Role	Description
---------	------	-------------

MonoBehaviourExtensions	Utility	Adds reflection-based property access to Unity components.
TryGetProperty	Method	Tries to retrieve a property value by name and type at runtime.
IPlayerMovementState	Interface	Defines what movement state information is expected.
PlayerMovementStateAdapter	Adapter	Reads and exposes those movement states from any component.
target	Variable	Stores the inspected component inside the adapter.
IsGrounded / IsMoving / IsCrouching / IsRunning	Properties	Reflectively check for each movement state condition.

On Input System Event

Overview

This system provides a **universal input event dispatcher** for games using Unity's **Input System**.

It allows any script to listen to input actions (for example, "Jump", "Move", "Fire") without directly managing state changes like *pressed*, *held*, or *released*.

Each script or component that uses this dispatcher acts as an **owner**.

This ownership system ensures that multiple scripts can share the same input action safely — each one receiving only its own callbacks.

The dispatcher automatically handles transitions between input states (press → hold → release), and it works with **any type of input value**, such as:

- Numbers (for example, trigger pressure as float).
- 2D or 3D vectors (for example, movement direction).
- Boolean values (for example, button pressed).

Core Concept

Imagine this system as a "hub" that monitors every input action continuously.

Each time an input changes, the hub detects whether it was just pressed, is being held, or has been released.

Then, it calls the correct functions (callbacks) registered by each script that owns that action.

It is **generic**, meaning it can operate with different data types for the input value.

Main Elements

1. Static Dispatcher

This is the global manager that keeps track of all registered input actions and their owners. It doesn't need to be created manually — it exists automatically when you use it.

2. State Class

Each input action has its own internal state container.

This class holds all the runtime information related to one specific input.

Variables Inside the State

- **action** – The input action being monitored (for example, “Jump” or “Move”).
- **lastValue** – The most recent value read from that input (for example, the joystick direction).

Callback Lists

Each of these lists contains a set of registered reactions (callbacks), grouped per owner:

- **OnHold** – Functions that are called continuously while the input is held down.
- **OnPressed** – Functions that are called once when the input transitions from released to pressed.
- **OnReleased** – Functions that are called once when the input transitions from pressed to released.

Each callback entry contains:

1. The **owner** that registered it (the script or object that listens).
2. The **callback function** to execute.
3. An **enabled condition**, a small function that decides whether the owner is currently active.

Owner Tracking

- **ownerWasHeld** – A record of which owners were holding the input in the previous frame, allowing the system to detect transitions and simulate releases when an owner becomes inactive.

3. The Update Routine

This is the heartbeat of the dispatcher.

It runs automatically after every input system update, meaning once per frame.

What it does:

1. Reads the current value of the input (for example, joystick position or button state).
2. Checks if this value represents a *non-zero* or *active* state.
3. For every owner:
 - Checks if that owner is currently enabled using its predicate (for example, whether the player is active).
 - If the owner just pressed the button, invokes the “pressed” callbacks.
 - If the button is still being held, invokes the “hold” callbacks.
 - If the button was released, invokes the “released” callbacks.

4. Updates the *last known value* for that input.

This mechanism ensures that pressed, hold, and released events are consistently detected, even if multiple owners are listening.

4. Helper Function: **GetEnabledForOwner**

This function looks through all the callback lists to find the specific enabled predicate associated with a particular owner.

It allows the system to know whether that owner is currently allowed to receive callbacks. If no predicate is found, the owner is assumed to be active by default.

5. Helper Function: **IsNonZero**

This function determines if an input value should be considered *active*.

Examples:

- For a float, any value above a small threshold (like joystick tilt) counts as active.
- For a vector, any non-zero direction means movement is active.
- For a boolean, “true” means pressed.
- For a quaternion, anything other than the identity rotation is active.

This function ensures consistency for different data types.

6. Helper Function: **UnregisterAll**

Removes every callback belonging to a specific owner from all lists, and clears that owner’s held state.

This is used when a component is destroyed or no longer wants to listen for input.

Registration System

The dispatcher provides several public functions that allow other scripts to register their own reactions to input events.

Each registration function requires three key elements:

- The **input action** to listen to.
- The **owner**, which identifies which script registered the callback.
- The **callback function**, which defines what should happen when the event occurs.

Available Registration Methods

1. **RegisterHold** – Adds a function that is repeatedly called while the input is active.

2. **RegisterPressed** – Adds a function that is called once when the input becomes active.
3. **RegisterReleased** – Adds a function that is called once when the input returns to inactive.
4. **UnregisterAll** – Removes all callbacks for a specific owner.
5. **Clear** – Completely removes an input action from the system, including every owner.

Each registration can include an **enabled predicate**, allowing conditional input handling. For instance, a player script might only receive input if the character is alive or not paused.

Automatic State Management

Whenever a new input action is used, the dispatcher automatically creates a state for it. It also connects its update method to Unity's input system update event, ensuring the state refreshes every frame.

This connection happens only once per action, no matter how many scripts register it.

Configuration Wrapper (Fluent Interface)

This secondary helper class provides a *fluent interface* for registering multiple events more cleanly.

It allows developers to chain setup calls in a readable format.

Example conceptually:

Configure "Jump" action for this player:

When pressed → call Jump()

When released → call StopJump()

Fields in the Config Wrapper

- **action** – The input being configured.
- **owner** – The script or component that owns these callbacks.
- **enabled** – The predicate that determines whether callbacks should run for this owner.

Methods

1. **OnHold** – Registers a callback for when the input is continuously held.
2. **OnPressed** – Registers a callback for when the input is first pressed.
3. **OnReleased** – Registers a callback for when the input is released.
4. **Dispose** – Removes all callbacks related to this owner and action.

Each method returns the configuration itself, allowing the calls to be chained together smoothly.

How It All Works Together

1. A game script defines an input action, such as “Jump”.
2. The script creates a configuration for that action using the dispatcher.
3. It registers functions for when the action is pressed, held, and released.
4. The dispatcher continuously checks the input’s value each frame.
5. Whenever the input changes, the dispatcher invokes the appropriate callbacks for that owner.
6. If the owner is disabled or destroyed, the dispatcher automatically stops invoking callbacks and can clean up all references.

Advantages

- **Universal:** Works with any data type (float, vector, bool, etc.).
- **Owner-based:** Multiple scripts can safely listen to the same input.
- **Automatic:** No need to manually manage input state changes.
- **Flexible:** Developers can define when callbacks are active using predicates.
- **Clean API:** Fluent configuration makes it easy to set up multiple event types in a single line.
- **Safe Cleanup:** Owners can unregister all their callbacks instantly.

Practical Usage Scenario

- A player controller script wants to jump when the button is pressed.
- A UI controller also listens to the same button to trigger a visual flash.
- Both can register to the same input action without overwriting each other.
- The player script’s callbacks are active only while the player is alive.
- The UI script’s callbacks are active only while the menu is open.
- The dispatcher ensures each receives its input events independently, respecting their conditions.

Summary Table

Element	Role	Description
states	Global dictionary	Stores all actions being monitored and their runtime data.
State	Nested class	Manages callbacks and transitions for a single input action.
OnHold / OnPressed / OnReleased	Lists	Store per-owner callbacks for hold, press, and release.
ownerWasHeld	Map	Tracks whether an owner was holding the input last frame.
Update	Function	Processes all callbacks based on input transitions each frame.

IsNonZero	Function	Determines if an input value counts as "active."
UnregisterAll	Function	Removes all callbacks for a given owner.
WithAction	Method	Creates a configuration wrapper for easy setup.
OnInputSystemEventConfig	Helper class	Provides a fluent interface for chaining event registrations.
Dispose	Method	Cleans up callbacks tied to an owner and action.

Action Block Timer

Purpose

This utility manages temporary action blocking in a game. It prevents a specific action (such as crouching, jumping, or attacking) from being triggered for a short period. This is useful to avoid repeated or unintended inputs, ensuring smooth and controlled gameplay.

Variables

1. IsBlocked

- A flag indicating whether the action is currently blocked.
- While true, the action cannot be executed.

2. timer

- Tracks the remaining time until the block expires.
- Decreases over time until it reaches zero.

3. duration

- The fixed length of time the action will be blocked when activated.
- This value is set when the utility is created and does not change.

Methods

1. Constructor (Initialization)

- Sets the block duration.
- Initializes the state so that the action starts unblocked.
- Initializes the timer to zero.

2. Activate

- Starts the block.
- Sets the IsBlocked flag to true.
- Resets the timer to the full duration.
- After calling this, the action is temporarily prevented from being executed.

3. Update

- Called repeatedly, typically once per frame in a game loop.
- Reduces the remaining timer based on the time elapsed since the last update.
- Automatically unblocks the action when the timer reaches zero by setting IsBlocked to false.

4. Reset

- Immediately ends the block.
- Sets IsBlocked to false and clears the timer.

- This allows the action to be executed right away without waiting for the timer.

Usage

1. Creating a Block Timer

- Create a new instance of the utility and specify the duration of the block.
- Example: For a crouch action, you might set a block of 0.15 seconds to prevent rapid toggling.

2. Activating the Block

- When the player performs the action (like pressing crouch), call Activate.
- The action will now be temporarily blocked.

3. Updating Each Frame

- Call Update in every frame of the game loop.
- This decreases the timer and automatically unblocks the action when the time runs out.

4. Checking if Action is Blocked

- Use IsBlocked to determine if the action is still unavailable.
- If true, ignore the player input for that action.

5. Resetting the Block

- If needed, you can manually reset the block to allow immediate action.

Summary of Behavior

- When activated, the utility prevents an action from occurring for a fixed period.
- The timer decreases automatically every frame.
- Once the timer expires, the action becomes available again.
- It provides a simple mechanism to avoid spamming actions in a game, improving gameplay control and responsiveness.

Player Jump Controller

Overview

The PlayerJumpController is a self-contained jump management system designed to control a character's jumping behavior using a physics body (Rigidbody).

It supports **multiple jumps**, **coyote time** (a short grace period after leaving the ground where jumping is still allowed), and a **cooldown after jumping** to prevent incorrect ground detection.

It is meant to be **instantiated and updated automatically**, and can be safely destroyed when no longer needed.

Main Components

Target Rigidbody Provider

A function (targetRigidbody) that returns the physics body to which the jump force will be applied.

This allows flexibility — for instance, it can dynamically point to any Rigidbody from another script.

◆ Ground Check Provider

A function (isGrounded) that returns whether the character is currently touching the ground. This check is provided externally, so the jump system does not need to handle collision detection itself — it just uses the result.

◆ Maximum Jumps Provider

A function (maxJumps) that defines how many jumps the character is allowed to perform before landing again.

If no value is provided, the default is **1**, meaning only a single jump.

This makes double or triple jumps possible simply by returning higher numbers.

◆ Coyote Time Provider

A function (coyoteTime) that specifies how long, in seconds, the player can still jump after leaving the ground.

If not set, the default value is **0**, meaning no coyote grace period.

⚡ Internal Runtime Variables

Variable	Purpose
jumpCount	Counts how many jumps have been used since the last time the player touched the ground.
lastGroundedTime	Stores the last time (in fixed time) the player was on the ground, to calculate coyote time.
wasGroundedLastFrame	Keeps track of whether the player was grounded in the previous physics frame, to detect takeoff and landing transitions.
jumpCooldownTime	Sets a temporary period after jumping during which the system ignores ground contact to prevent false grounded detection.
groundedIgnoreDurationAfterJump	A constant defining the cooldown duration (0.1 seconds by default).
coyoteJumpUsed	Flags whether the coyote jump (the grace jump) has already been used.
isInCoyoteWindow	Indicates if the player is still within the grace window where a coyote jump can occur.

Construction Process

When the controller is created:

- It receives the four functional parameters (rigidbody provider, grounded state, max jumps, and coyote time).
- If optional ones are not given, it substitutes them with default behaviors.
- It subscribes automatically to a **FixedUpdate broadcaster**, meaning it will be updated every physics frame without manual management.

This design makes it ideal for plug-and-play use in larger systems.

How Jump Execution Works (OnJump)

When a jump input is triggered:

1. **Validation** – The controller first checks if:
 - The Rigidbody exists.
 - The game is not paused (`Time.timeScale > 0`).
 - There is at least one jump available.
2. **Jump Permission Rules:**
 - Jump is allowed if the character is currently grounded.
 - Jump is allowed if still inside the coyote time window and the coyote jump hasn't been used.
 - Jump is allowed if the player still has available jumps left (e.g., double jump).
3. **Force Application:**
 - The character's vertical velocity is reset so that the jump is consistent regardless of falling or moving upward.
 - An **impulse force** is applied upward through the Rigidbody.
4. **Post-Jump Cooldown:**
 - The system records the time until which the grounded detection should be ignored to avoid triggering early ground contact (useful when hitting the ceiling or the ground right after jumping).
5. **Coyote Tracking:**
 - If this jump happened during coyote time, it marks that the coyote jump was used to avoid another one in the same air phase.
6. **Jump Counter Update:**
 - The total jump count increases by one.

Continuous Physics Update (Update)

This internal routine runs every **FixedUpdate** frame and ensures the controller maintains consistent jump logic.

It manages transitions, timers, and state resets automatically.

Step-by-step:

1. Ground Check

It retrieves the current grounded state from the provided function.

If the current time is still inside the post-jump cooldown, it temporarily ignores ground detection.

2. Landing Detection

If the player **was in the air** last frame and **is now grounded**, it:

- Resets the jump count to 0.
- Resets the coyote jump flag.
- Updates the last grounded time.

3. Leaving Ground

If the player **was grounded** and **is now in the air**, it:

- Starts the coyote time countdown.
- Marks that a coyote jump is available.

4. Coyote Time Calculation

It checks whether the current time minus the last grounded time is still within the allowed coyote time limit.

5. Anti-Infinite Jump Protection

If the coyote time expires and the player has not jumped yet, it increments the jump count to ensure no infinite air jumps are possible.

Disposal

There is a static Dispose method that safely removes the controller from the physics update loop and nullifies the reference.

This is important for avoiding memory leaks or accidental updates after destruction.

Typical Usage Example (Conceptually)

1. A player script provides:
 - A function that checks if the character is grounded.
 - A function returning its Rigidbody.
2. It creates a new instance of this jump controller.
3. When the player presses the jump button, it calls OnJump(5f) or another desired force.
4. The controller applies force, manages jumps, and prevents false ground detections automatically.
5. Once destroyed, the system unsubscribes from the physics broadcaster.

Summary of Functionality

Feature	Description
Multi-Jump	Allows multiple jumps in mid-air, limited by a configurable maximum.
Coyote Time	Provides a small grace period after stepping off a ledge where jumping is still possible.
Cooldown After Jump	Prevents ground misdetection immediately after jumping.

Auto Reset on Landing	Jump count resets as soon as the player touches the ground.
Automatic Update Handling	The controller updates automatically each physics frame without needing MonoBehaviour inheritance.
Safe Disposal	Can be unregistered and destroyed cleanly at any time.

Player Move Controller

Overview

The PlayerMoveController is a **movement management system** for a character that uses a physics body (Rigidbody).

It allows the player to move in the horizontal plane (X and Z axes), with **configurable speed** and **optional obstacle blocking logic**.

It is **modular**, meaning it can be instantiated with custom functions for speed, target Rigidbody, and obstacle checks.

Main Components

Target Rigidbody Provider

A function that returns the physics body (Rigidbody) of the character.
The controller applies movement directly to this Rigidbody.

Speed Provider

A function that returns the movement speed.

If not provided, the default speed is **3 units per second**.

This allows dynamic adjustment of speed, for example, when sprinting or slowing down.

Obstacle Check Provider

A function that determines if movement should be blocked, e.g., when the character hits a wall.
If not provided, it always allows movement.

When active, the controller stops horizontal motion entirely while preserving vertical velocity (falling/jumping is unaffected).

Constructor

When the controller is created:

1. It stores references to the Rigidbody, speed function, and obstacle function.
2. If optional functions are not provided, it substitutes them with default behaviors:
 - Speed defaults to 3.

- Obstacle check defaults to false.

This design ensures the controller works immediately without requiring full configuration.

Main Methods

1. Move Method (OnMove)

Purpose: Moves the character in the horizontal plane according to a normalized direction vector.

Step-by-step:

1. **Check for Obstacles**
If the obstacle function returns true, it stops movement and exits early.
2. **Retrieve Rigidbody**
Gets the current Rigidbody from the provider. If null, the function exits.
3. **Determine Speed**
Retrieves the current movement speed from the speed provider.
4. **Calculate Movement Vector**
Normalizes the input direction and multiplies it by speed to determine XZ movement.
5. **Apply Movement**
 - Preserves vertical velocity (Y-axis) so that jumping or falling is unaffected.
 - Sets Rigidbody velocity to the new XZ vector plus existing Y velocity.

2. Stop Method (OnStop)

Purpose: Stops all horizontal movement immediately.

Step-by-step:

1. Retrieves the Rigidbody.
2. Reads current velocity.
3. Sets X and Z components of velocity to zero while keeping Y velocity intact.

This is useful when the player collides with an obstacle, or when movement input is released.

3. Dispose Method

Purpose: Clears the reference to the controller.

This is a simple cleanup utility for memory safety, though no subscriptions or updates are tied to this controller.

Usage Concept

1. Instantiate the Controller

Provide functions for:

- Returning the Rigidbody of the player.
- Returning the movement speed (optional).
- Checking if movement should be blocked (optional).

2. Call Movement

Every frame, pass a **normalized XZ input vector** to OnMove to move the player.

3. Stop Movement

If no input is detected, call OnStop to halt horizontal motion.

4. Cleanup

Optionally call Dispose when the player or game object is destroyed.

Summary of Features

Feature	Description
XZ Movement	Moves the character horizontally while preserving vertical velocity.
Speed Control	Can dynamically adjust speed using a provider function.
Obstacle Handling	Can block movement when an obstacle is detected.
Stop Mechanism	Immediately halts horizontal motion without affecting vertical motion.
Memory Safe	Provides a dispose method for cleanup, preventing lingering references.

This controller works very well in combination with **jump controllers** or other physics-based systems because it **only manipulates horizontal motion** while letting vertical motion (falling or jumping) remain under the control of other systems.

Player Rotation Controller

Overview

The PlayerRotationController is a **system for smoothly rotating a game object** in 3D space based on directional input.

It focuses on **horizontal rotation**, meaning the object rotates in the XZ plane according to player input or movement direction.

The rotation is **smoothed over time** to avoid abrupt changes.

Main Components

Target Transform

A function that provides the transform to be rotated.

This is the object that will visually turn to face the movement direction.

◆ Direction Provider

A function that returns a **2D direction vector** (X and Y values).

- X maps to the world X-axis.
- Y maps to the world Z-axis.
This vector usually comes from input or movement logic and determines which direction the player wants to face.

◆ Rotation Speed

A function returning a multiplier for how fast the object rotates toward the target direction.

- Default speed is **10 units** if not provided.
Higher values make rotation faster and more responsive, while lower values create a slower, smoother turning effect.

◆ Magnitude Threshold

A function returning the minimum length of the directional vector required to trigger rotation.

- Default is **0.01**.
This prevents the controller from rotating when the input is extremely small or near zero (for example, joystick drift).

Constructor

When the controller is created:

1. Stores references to the target transform, direction, speed, and magnitude functions.
2. Sets defaults for speed and magnitude if not provided.
3. Registers an **update method** to run on each physics update frame (FixedUpdate).
This ensures that rotation is applied smoothly in sync with the physics engine.

Main Methods

1. Update Rotation

Purpose: Smoothly rotates the target object toward the desired direction.

Step-by-step:

1. **Read the direction vector** from the direction provider.
2. **Check magnitude:** Only rotate if the input vector is longer than the minimum threshold.

3. **Convert 2D to 3D:** Maps the 2D X/Y vector to a 3D X/Z vector in the horizontal plane.
4. **Calculate target rotation:** Determines the rotation needed for the object to face the input direction.
5. **Smooth rotation:** Gradually interpolates the current rotation toward the target rotation using a smoothing factor based on speed and fixed delta time.

This method ensures that even if input changes suddenly, the rotation will **transition smoothly** instead of snapping instantly.

2. Dispose

Purpose: Cleans up the controller.

- Unregisters the update method from the physics update loop.
- Clears the reference to the controller to prevent memory leaks.

This is important when the object is destroyed or the controller is no longer needed.



Usage Concept

1. Instantiate the Controller

Provide functions for:

- Returning the object's transform.
- Returning the current input direction (2D vector).
- Optional rotation speed.
- Optional magnitude threshold.

2. Update Automatically

On each physics frame, the controller reads the direction and rotates the object smoothly toward it.

3. Stop / Cleanup

When the object is destroyed or rotation control is no longer needed, call the dispose method to unregister the update.



Summary of Features

Feature	Description
Smooth Rotation	Uses interpolation to avoid snapping and sudden turns.
XZ Plane Rotation	Only rotates horizontally based on input direction.
Input Threshold	Prevents rotation when input is near zero.
Configurable Speed	Allows fast or slow rotation depending on gameplay needs.
Automatic Update	Runs each fixed physics frame for consistent motion.
Safe Disposal	Can unregister from updates to prevent memory leaks.

This controller is particularly useful for **third-person characters, top-down games, or any Rigidbody object** that needs to face the direction of movement smoothly.

It can also be **combined with movement and jump controllers** for full character control.

Player Speed Control

Overview

The PlayerSpeedControl manages a **player's movement speed** along with an **optional stamina system**. It allows the player to switch between normal walking speed and faster running speed, depending on stamina availability. The script also automatically handles **stamina depletion when running** and **regeneration when idle or walking**.

Main Components

Delegates / Functions

These are functions provided during construction to configure behavior dynamically:

1. **Normal Speed** – Returns the player's normal walking speed.
2. **Fast Speed** – Returns the running speed.
3. **Maximum Stamina** – Returns the highest stamina value the player can have.
4. **Minimum Stamina to Run** – Returns the threshold required to allow running.
5. **Stamina Depletion Rate** – Returns how fast stamina decreases while running.
6. **Stamina Recovery Rate** – Returns how fast stamina regenerates when not running.
7. **Is Moving** – Returns whether the player is currently moving.

These functions allow the controller to adapt to external variables like buffs, debuffs, or dynamic player states.

Private Fields

- **useStamina** – Indicates whether the stamina system is active.
- **currentStamina** – Current stamina value of the player.
- **subscribedToUpdate** – Tracks whether the controller is actively subscribed to physics updates.
- **isFast** – Tracks if the player is currently in running mode.
- **haveEnoughStamina** – Indicates if the player has enough stamina to run.

Public Fields

- **resultingVelocity** – The final speed of the player based on whether they are running or walking.
- **hasStamina** – True if the player is running and has sufficient stamina.
- **staminaPercentage** – Current stamina expressed as a percentage of maximum stamina for UI or logic purposes.

Constructors

The controller provides **two modes of operation**:

1. **Without stamina** – Simply switches between normal and fast speed, ignoring stamina.
2. **With stamina** – Enables stamina system with depletion, regeneration, and running restrictions based on stamina. Default values are provided if specific rates or thresholds are not supplied.

The stamina-enabled constructor also **subscribes the update logic** to run every physics frame.

Public Methods

1. Start Running

- Attempts to switch the player to running mode.
- Checks if stamina system is active and whether the player has enough stamina.
- Updates the resulting velocity according to whether running is allowed.

2. Stop Running

- Forces the player to stop running.
- Resets the resulting velocity to normal walking speed.

Update Logic

This is the core of the stamina system, executed every physics frame:

1. **Check running conditions**: Only deplete stamina if running and moving.
2. **Deplete stamina**: Subtracts stamina based on depletion rate and time elapsed.
 - If stamina reaches zero, running stops automatically.
3. **Regenerate stamina**: If the player is not running or moving, stamina regenerates over time.
 - Once it exceeds the minimum threshold, running becomes available again.
4. **Update running state**: Sets hasStamina based on whether the player is running and has enough stamina.
5. **Update UI values**: Calculates stamina percentage for display or further logic.

This ensures **smooth stamina behavior**, automatically switching between running and walking depending on the player's stamina.

Subscriptions / Disposal

- The update logic is connected to the physics update loop via an internal subscription.

- A **dispose method** exists to safely unsubscribe the controller and prevent memory leaks when it's no longer needed.

Usage Concept

1. Without stamina system

- Provide only normal and fast speed functions.
- Call StartRunning and StopRunning to switch speeds.

2. With stamina system

- Provide functions for normal speed, fast speed, movement state, and optionally stamina configuration.
- StartRunning will automatically consider stamina.
- Update will handle stamina depletion and regeneration automatically.
- resultingVelocity always reflects the current effective movement speed.
- staminaPercentage can be used for UI or logic checks.

Summary of Features

Feature	Description
Running/Walking Speed	Automatically chooses speed based on stamina and input.
Stamina Depletion	Reduces stamina while running.
Stamina Regeneration	Restores stamina when walking or idle.
Automatic Running Restriction	Stops running when stamina is too low.
Stamina Threshold	Prevents running unless minimum stamina is reached.
Resulting Velocity	Always provides the effective movement speed.
Stamina Percentage	Exposes current stamina for UI purposes.
Safe Update Subscription	Handles physics updates automatically and can be disposed safely.

This controller is ideal for **any player character that can run**, with realistic stamina management and automatic speed adjustments.

It integrates easily with **movement and rotation controllers** to form a complete player control system.

Player Utils

Overview

The PlayerUtils class is a **utility helper** for player movement and collision handling. Its primary role is to **perform vector transformations**, which is useful for rotating movement directions, calculating slopes, or adjusting player input relative to orientation.

Currently, this utility focuses on **rotating a 2D vector** by a specified angle.

Main Components

◆ Method: Vector Rotation

Purpose: Rotate a 2D directional vector by a specified angle in degrees.

Inputs:

1. **Direction** – A two-dimensional vector representing the original movement direction (X and Y components, where Y is typically mapped to Z in 3D movement).
2. **Rotation** – A rotation angle in degrees, which will rotate the vector clockwise around the origin.

Process:

- Converts the input angle from degrees to radians because trigonometric calculations use radians.
- Pre-calculates the cosine and sine of the rotation angle to improve performance.
- Uses the **2D rotation formula** to compute the new vector components:
 - The new X component is calculated using the formula: $(\text{original X} * \cos) - (\text{original Y} * \sin)$.
 - The new Y component is calculated using the formula: $(\text{original X} * \sin) + (\text{original Y} * \cos)$.
- Returns the **rotated 2D vector**, which can then be used for movement, aiming, or directional adjustments.

◆ Usage

This utility is useful in scenarios such as:

1. **Player movement relative to camera rotation**
 - You have a player input vector for movement (forward, backward, sideways) and want it rotated to match the camera orientation.
2. **Calculating projectile or object trajectories**
 - Rotate a direction vector to fire bullets or projectiles at an angle.
3. **Slope or angled movement adjustments**
 - When handling slopes, you may rotate movement vectors to align with the slope's angle.

Example Concept:

- Suppose the player input is moving forward (0,1) but the character is rotated 45 degrees relative to the world.
- By calling the rotation utility with 45 degrees, the resulting vector will move diagonally, correctly aligned with the character's orientation.

◆ Key Characteristics

- **Static method** – Can be called without creating an instance of the utility class.

- **Mathematical precision** – Uses radians for trigonometric functions to ensure accurate rotation.
- **General-purpose** – Can be used for any 2D directional adjustments in movement, aiming, or physics calculations.

In short, PlayerUtils is a lightweight helper that simplifies **vector rotation tasks** for player movement and other gameplay mechanics requiring directional transformation. It is intended to be **reusable** wherever 2D vector rotation is needed.

Physics Update Broadcaster

Overview

PhysicsUpdateBroadcaster is a **singleton component** that provides a centralized way to broadcast Unity's standard lifecycle events—**Update**, **FixedUpdate**, and **LateUpdate**—through **static events**. This allows other scripts to subscribe to these events without needing to attach multiple MonoBehaviours.

Additionally, it manages a **list of BoxCollisionSensor components** to draw **gizmos** in the editor or during runtime, depending on the sensor's configured display mode.

The broadcaster ensures that there is **only one instance** in the game, which persists across scenes.

Main Components

◆ Singleton Instance

- **Purpose:** Ensures that only one instance of the broadcaster exists.
- **Behavior:**
 - When a new instance is created, it checks if an existing one already exists.
 - If a duplicate exists, it destroys itself.
 - The surviving instance persists across all scene loads.

◆ Static Events

The broadcaster provides three main static events that other scripts can subscribe to:

1. **OnUpdate** – Triggered every frame during Unity's normal Update loop.
2. **OnFixedUpdate** – Triggered every physics frame during FixedUpdate.
3. **OnLateUpdate** – Triggered every frame at the end of the Update cycle.

Usage: Any script can subscribe a method to these events to receive lifecycle updates without needing its own MonoBehaviour Update method. This is particularly useful for controllers and systems that are not attached to GameObjects.

◆ Registered Sensors

- The broadcaster maintains a **list of BoxCollisionSensor components**.
- Sensors provide information about their **position, rotation, size, and color**.
- Depending on the **gizmo drawing mode**, the broadcaster decides whether to draw the sensor as a wireframe cube in the scene view:
 - **Always** – The sensor gizmo is always drawn.
 - **SelectedOnly** – The sensor gizmo is drawn only when the sensor's target object is selected in the editor.
 - **None** – The sensor gizmo is never drawn.

◆ Gizmo Drawing Methods

1. **OnDrawGizmos** – Called by Unity to render gizmos in the scene.
 - Iterates through all registered sensors.
 - Checks their drawing mode and target selection.
 - Calls a helper to draw the actual wireframe.
2. **DrawSensorGizmo** – Responsible for the actual rendering of a wireframe cube based on the sensor's:
 - **Position** – Where the box is located.
 - **Rotation** – Orientation of the box.
 - **Size** – Width, height, and depth.
 - **Color** – Color of the cube lines.

◆ Sensor Management

- **AddSensor** – Adds a BoxCollisionSensor to the internal list if it's not already present.
- **RemoveSensor** – Removes a BoxCollisionSensor from the internal list.

This allows dynamic registration and unregistration of sensors during gameplay or editor interaction.

◆ Initialization

- The broadcaster ensures a single persistent instance exists **automatically at runtime** using a runtime initialization method.
- If no instance exists, it creates a new GameObject and attaches the broadcaster to it, guaranteeing that events and gizmo drawing will always work without manual setup.

⚡ Usage Example

1. **Subscribing to events:**

- A script can subscribe its own update methods to the broadcaster:
 - For physics logic: subscribe to OnFixedUpdate.
 - For frame-based logic: subscribe to OnUpdate.
 - For post-frame adjustments: subscribe to OnLateUpdate.

2. Managing sensors:

- Attach a BoxCollisionSensor component and register it using AddSensor.
- The broadcaster will automatically handle drawing the sensor as a gizmo according to its configured mode.

3. Persistence across scenes:

- Once initialized, you do not need to manually add it to each scene.
- Other systems can safely subscribe to events regardless of scene changes.

◆ Key Characteristics

- **Singleton:** Only one instance exists and persists across scenes.
- **Static Events:** Provides a centralized mechanism for broadcasting Unity lifecycle events.
- **Gizmo Management:** Handles rendering of sensors dynamically based on their display mode.
- **Editor Support:** Integrates with the Unity Editor to only draw certain gizmos when selected.
- **Automatic Setup:** No manual scene setup is required; it creates itself at runtime if missing.

In short, PhysicsUpdateBroadcaster acts as a **central event hub** for physics and update loops while also managing the **visual debugging of collision sensors**, making it an essential backbone for player and physics systems in the game.

Camera State Manager

✖ Overview

CameraStateManager is a Unity component designed to **control the active state of a camera dynamically**. Its main purpose is to enable or disable the camera based on:

- Player input (button press or input action)
- The active state of certain GameObjects (like menus, UI panels, or triggers)
- Manual control through other scripts

It supports multiple **update modes** and automatically handles **cursor locking and visibility** when toggling the camera.

⚙ Main Components

◆ Update Type

This defines how and when the camera state is updated:

1. **ManualToggle** – Camera is controlled only via explicit calls from other scripts.
2. **InputEvent** – Camera toggles when the player presses a configured input action.
3. **Update** – Camera state is automatically evaluated every frame in Unity's Update loop.
4. **FixedUpdate** – Camera state is evaluated at fixed physics intervals for consistent timing.

◆ Input Settings

- **Toggle Action** – An input action (like a button press) used to toggle the camera on or off.
- The script subscribes to the input event only if the update type is set to InputEvent.

◆ Activation Objects

- A list of GameObjects that influence the camera state.
- If **any object in the list is active**, the camera will be disabled.
- The camera will re-enable itself when all objects are inactive.
- The system uses a **0.1-second threshold** to prevent rapid toggling if objects change state quickly.

◆ Camera State Variables

- **isCameraActive** – Tracks whether the camera is currently active.
- **lastActivationState** – Stores the previous frame's activation state of the monitored objects.
- **lastUpdateTime** – Timestamp of the last state evaluation.
- **activeDuration** – Time that activation objects have continuously been active.
- **inactiveDuration** – Time that activation objects have continuously been inactive.

These variables help the script avoid flickering and ensure smooth transitions.

◆ Events

- **OnEnable** – Triggered when the camera is activated. Useful for triggering UI, sounds, or other systems.
- **OnDisable** – Triggered when the camera is deactivated.

This allows other systems to react when the camera state changes.

◆ Debug Mode

- When enabled, the script prints messages to the console whenever the camera state changes or when the component initializes.

- This is helpful for development and troubleshooting.

◆ Methods and Usage

1. **Awake**

- Checks if the toggle input is assigned.
- Warns and disables the component if the input is missing.
- Useful for catching configuration issues early.

2. **Start**

- Initializes the camera to active by default.
- Registers the input event if using InputEvent mode.
- Ensures the camera can respond to player input immediately.

3. **OnDestroy**

- Cleans up input event subscriptions to prevent memory leaks.

4. **Update / FixedUpdate**

- Depending on the update type, evaluates the camera state using the activation objects every frame (Update) or at fixed intervals (FixedUpdate).

5. **SetCameraState(active)**

- Activates or deactivates the camera.
- Updates cursor lock and visibility: locks the cursor when active, unlocks when inactive.
- Triggers the corresponding UnityEvent (OnEnable or OnDisable).
- Handles debug logging if enabled.

6. **UpdateCameraStateFromObjects**

- Checks the list of monitored GameObjects.
- If any object is active for longer than 0.1 seconds, the camera is disabled.
- If all objects are inactive for longer than 0.1 seconds, the camera is enabled.
- Avoids flickering when objects rapidly toggle on/off.

7. **IsAnyActivationObjectActive**

- Helper method that checks if **any** object in the monitored list is active.
- Returns true if at least one object is active, otherwise false.

⚡ Usage Scenarios

1. **Menu or UI Handling**

- Automatically disable the camera when a menu is open.
- Reactivate when the menu closes.

2. **Input Toggle**

- Use a key or controller button to toggle between camera on/off.

3. **Manual Control**

- Other scripts can directly call SetCameraState to enable or disable the camera based on game logic.

4. **Smooth Automatic Updates**

- Use Update or FixedUpdate mode to continuously monitor certain objects and switch the camera state accordingly.

◆ Summary

CameraStateManager provides a **flexible and centralized system** to control camera activation, combining:

- Manual, input-driven, or automatic modes
- Object-based activation/deactivation
- Event hooks for other systems
- Cursor management
- Flicker-free state transitions

It is particularly useful for games where the camera needs to respond dynamically to menus, UI, cutscenes, or player input.

Footstep Sound Controller

✚ Overview

FootstepSoundController is a Unity component responsible for **playing footstep and landing sounds dynamically**, adapting to the player's movement state. It uses reflection to read movement properties from any player controller that implements the required interface. This allows it to work with different movement systems without being tied to a specific implementation.

It handles:

- Walking, running, crouch-walking, and crouch-running footstep sounds
- Landing sounds when the player touches the ground after jumping or falling
- Dynamic timing and pitch variation to make sounds feel natural

⚙ Main Components

◆ Movement States

The script defines different movement states:

1. **None** – Player is idle; no footstep sounds play.
2. **Walking** – Normal walking.
3. **Running** – Normal running.
4. **CrouchWalking** – Walking while crouched.
5. **CrouchRunning** – Running while crouched.

These states determine which sounds are played and the timing intervals between footsteps.

◆ Serialized Fields (Configurable in Inspector)

- **Player Controller** – Reference to the movement controller component. Must expose properties like "IsMoving", "IsRunning", "IsCrouching", and "IsGrounded".
- **AudioSource** – Component responsible for playing the sounds.

- **AudioMixerGroup** – Optional, allows routing the AudioSource through a mixer for volume control or effects.
- **Footstep Volume** – Base volume for all footstep and landing sounds.
- **Minimum and Maximum Pitch** – Used to randomize pitch for variation in footsteps.

Timing Settings:

- Walk interval, run interval, crouch-walk interval, crouch-run interval – define the time between footsteps for each movement state.
- Footstep clip lists – separate lists for normal and crouching movement sounds.
- Landing sounds – optional clips to play when the player lands on the ground.

◆ Private Fields (Runtime State)

- **playerState** – Adapter that uses reflection to read movement properties from the player controller.
- **currentInterval** – Time between footsteps based on the current movement state.
- **lastFootstepTime** – Time when the last footstep sound was played.
- **wasGroundedLastFrame** – Tracks whether the player was grounded last frame to detect landing events.

◆ Properties

Exposes access to:

- Player controller
- AudioSource
- AudioMixerGroup
- Footstep volume
- Pitch range
- Interval settings for walking, running, crouching
- Footstep and landing audio clips
- Landing sound toggle

These allow other scripts to adjust sound behavior at runtime.

◆ Methods and Usage

Awake

- Initializes the script and validates references.
- Sets up the playerState adapter to read movement properties via reflection.
- Configures the AudioSource (disables play-on-awake, sets volume, assigns mixer group).
- Prepares the timing for the first footstep to play immediately.

FixedUpdate

- Called each physics step.
- Checks if the player has landed and plays landing sounds if enabled.
- Determines the current movement state.
- Updates footstep timing and triggers sounds if enough time has passed.

HandleLandingSound

- Detects when the player lands on the ground after being airborne.
- Plays a random landing sound if enabled, otherwise plays a regular footstep.
- Updates the grounded state for the next frame check.

UpdateFootsteps

- Determines whether enough time has passed since the last footstep.
- Skips footstep playback if the player is not moving or not grounded.
- Plays a footstep corresponding to the current movement state (walking, running, crouching).

PlayFootstep

- Selects a random audio clip from the appropriate list based on the movement state.
- Uses pitch randomization to make repeated steps sound natural.
- Calls the AudioSource to play the sound.

PlayFootstepClipWithPitch

- Plays a single AudioClip with a randomized pitch between the minimum and maximum values.
- Ensures footsteps and landings do not sound repetitive.

GetMovementState

- Evaluates the player's movement flags (IsMoving, IsRunning, IsCrouching) and returns the correct movement state.
- Ensures that footstep timing and clip selection match the actual player behavior.

SetCurrentInterval

- Sets the interval between footsteps based on the current movement state.

- Prevents footsteps from playing when the player is idle.

Usage Scenarios

1. **Footstep Timing** – Automatically plays sounds at realistic intervals for walking, running, crouching, or crouch-running.
2. **Landing Sounds** – Adds immersion by playing a sound when the player hits the ground.
3. **Dynamic Variation** – Random pitch ensures footstep sounds do not become repetitive.
4. **Flexible Integration** – Works with any player controller implementing the required movement interface.
5. **Audio Mixing** – Can route all sounds through a mixer for global volume control or effects.

Summary

FootstepSoundController is a **dynamic and adaptable system** for adding realistic footstep and landing sounds. It automatically:

- Detects movement and grounded state
- Plays footstep sounds with appropriate timing for walking, running, and crouching
- Randomizes pitch for variation
- Supports optional landing sounds
- Works with any compatible player movement controller

This makes it suitable for first-person or third-person games where realistic audio feedback for movement is important.

Look Input Handler

Overview

LookInputHandler is a Unity component designed to **capture and manage camera look input** using the Unity Input System. It stores the player's look direction as a two-dimensional vector and allows enabling or disabling input dynamically. This is useful for controlling the camera in first-person or third-person games, while also being able to temporarily block input during cutscenes, menus, or special gameplay states.

Main Components

Serialized Fields

1. **Input Action Reference (lookAction)**
 - Represents the Input System action that captures the player's camera look input.
 - Typically linked to mouse movement or right stick on a gamepad.
2. **Look Direction (LookDirection)**
 - Stores the latest camera look input as a two-dimensional vector.
 - Updated continuously while input is active.

- Reset to zero when input is released or disabled.

◆ Private Fields

1. Look Input Event (**lookInputEvent**)

- Encapsulates the configuration and binding of the input event.
- Handles both continuous input (holding) and input release.

2. Playable Flag (**isPlayable**)

- Indicates whether input processing is currently enabled.
- Can be toggled at runtime to allow or block camera control.

◆ Properties

• Look Action

- Provides public access to the input action reference.
- Can be changed at runtime if you want to switch input schemes or devices.

◆ Methods and Usage

Awake

- Checks whether the input action reference is assigned.
- Issues a warning if the action is missing to prevent runtime errors.

Usage: Ensures the component is properly configured before the game starts.

Start

- Configures the input event to capture the look input.
- Binds two behaviors to the input:
 1. **On Hold** – Updates LookDirection while the input is active.
 2. **On Released** – Resets LookDirection to zero when the input stops.

Usage: Automatically handles continuous input updates and ensures the look vector always reflects the player's current intent.

OnDestroy

- Cleans up the input event when the object is destroyed.
- Prevents lingering event bindings that could cause memory leaks or unwanted behavior.

Usage: Ensures proper cleanup when switching scenes or disabling the component.

TogglePlayable

- Allows enabling or disabling input dynamically at runtime.
- When input is disabled, the look direction is immediately reset to zero.
- When input is enabled, look input resumes normally.

Usage: Can be used for scenarios such as:

- Disabling camera control when the player opens a menu.
- Locking the camera during a cutscene.
- Pausing input temporarily for special game mechanics.

⚡ Key Behavior

1. **Continuous Look Tracking** – The component constantly updates LookDirection while the player moves the mouse or right stick.
2. **Dynamic Input Control** – The camera input can be blocked and unblocked without affecting other systems.
3. **Zeroing Input on Release** – Automatically resets the vector to prevent drift when input stops.
4. **Integration Friendly** – Can be queried by any camera or player controller script to determine how the camera should rotate.

◆ Summary

LookInputHandler is a **flexible and robust solution for managing camera input** in Unity. It:

- Captures continuous look input
- Stores it as a vector for other systems to use
- Supports runtime enabling/disabling of input
- Automatically handles cleanup and input release

This makes it ideal for first-person or third-person games that require dynamic and responsive camera control while maintaining full flexibility over when input is processed.

Player Animation Controller 3D

✖ Overview

The PlayerAnimationController3D is a Unity component designed to manage **animation transitions for a 3D side-view player**. It uses **boolean parameters in the Animator** to ensure that only one animation state is active at a time. The active state is determined by the player's **movement, grounded, and crouching status**, keeping animations consistent with the player's behavior.

Main Components

◆ Animation States

The script defines several possible animation states internally:

1. **Stopped** – Player is idle on the ground and not crouching.
2. **Walking** – Player is moving on the ground without running or crouching.
3. **Running** – Player is moving faster (running) on the ground without crouching.
4. **Jumping** – Player is in the air and not crouching.
5. **Crouching** – Player is idle while crouching.
6. **Crawling** – Player is walking while crouching.
7. **CrawlingRunning** – Player is running while crouching.

Only **one state is active at any given time**, which prevents animation conflicts.

◆ Inspector Fields

1. **Player Controller Reference**
 - A reference to the script that provides movement and state information.
 - Required to read whether the player is moving, grounded, running, or crouching.
2. **Animator Reference**
 - Controls the player's animations.
 - The script updates boolean parameters to switch animation states.
3. **Animator Parameter Names**
 - Each animation state corresponds to a boolean parameter in the Animator:
 - stopped for idle
 - walking for walking
 - running for running
 - jumping for jumping
 - crouching for idle crouching
 - crawling for walking while crouched
 - crawlingRunning for running while crouched
4. **Current State**
 - Stores the active animation state during runtime for quick reference.
 - Updated whenever the state changes.

◆ Private Fields

1. **State Hashes**
 - Stores cached integer hashes of the Animator parameters for performance.
 - Hashes are used to set booleans efficiently rather than using string names repeatedly.
2. **Player State Adapter**
 - Provides access to the player's movement and status using an interface.
 - Determines if the player is moving, grounded, running, or crouching.

◆ Unity Lifecycle Methods

1. Awake

- Validates that the player controller and animator references are assigned.
- Initializes the adapter for player movement state.
- Caches the parameter hashes for all animation states.
- Sets the initial animation state to Stopped (idle).

Usage: Ensures the component is ready and that Animator updates are efficient from the start.

2. Update

- Every frame, determines the appropriate animation state based on the player's movement and status.
- Only triggers a state transition if the new state is different from the current state.
- Updates the Animator booleans: disables the old state and enables the new state.

Usage: Keeps animations synchronized with the player's movement in real time.

◆ Helper Method: DetermineState

- Evaluates the player's movement, grounded, and crouch flags to decide which animation state should be active.
- Covers all combinations: idle, moving, running, jumping, crouching, and crouch movement.
- Returns the current state if no changes are detected, preventing unnecessary updates.

Usage: Central logic that maps player status to animation states.

⚡ Key Behavior

1. **Single Active State** – Ensures that only one animation boolean is active at a time.
2. **Automatic Transitions** – Automatically switches animations when the player starts moving, jumps, crouches, or changes speed.
3. **Performance Efficient** – Uses cached parameter hashes instead of strings to set booleans in the Animator.
4. **Flexible** – Works with any player controller exposing the required movement properties.

◆ Summary

PlayerAnimationController3D is a **modular animation manager** that:

- Uses Animator booleans to control animations
- Automatically switches states based on player movement, crouching, and grounded status
- Avoids conflicts by ensuring only one animation is active at a time
- Efficiently manages state transitions with cached hashes

It is ideal for 3D side-view players in Unity, providing smooth and consistent visual feedback for player actions.

Stamina Monitor

Overview

The StaminaMonitor is a Unity component designed to **display and update a player's stamina in the UI**. It works with **any player controller** that exposes a public stamina percentage property. The script updates a slider and optional text to reflect the current stamina level and changes the slider color based on thresholds for empty, low, or normal stamina.

Main Components

Inspector Fields

1. **Player Controller Reference**
 - The component that provides stamina data.
 - Must have a public property exposing the stamina percentage.
 - The script reads this property via reflection, so it can work with any compatible controller.
2. **Max Stamina**
 - The maximum value used to normalize stamina for the slider.
 - Helps convert raw stamina to a 0–100 percentage scale.
3. **Minimum Stamina for Run**
 - Defines the low stamina threshold for color changes.
 - If stamina falls below this, the slider shows a low warning color.
4. **Stamina Slider Reference**
 - The UI slider representing current stamina.
 - The fill area's color is updated based on thresholds.
5. **Stamina Colors**
 - **Empty Stamina Color:** Color used when stamina reaches zero.
 - **Low Stamina Color:** Color used when stamina is below the running threshold but not zero.
 - **Normal Stamina Color:** Color used when stamina is above the low threshold.
6. **Stamina Text Reference**
 - Optional text element that displays the stamina percentage numerically.

Private Fields

1. **Slider Fill Image**
 - Reference to the fill area of the slider.
 - Allows the script to change the color dynamically.
2. **Minimum Stamina Color Threshold**
 - Precomputed normalized threshold for when the slider should switch to low stamina color.
3. **Player Stamina Adapter**
 - Interface that exposes the stamina percentage.

- Uses reflection to retrieve the value from any compatible player controller.

◆ Public Properties

- Expose references and thresholds to allow runtime modification.
- Includes player controller, slider, maximum stamina, colors, and text.

◆ Unity Lifecycle Methods

1. Start

- Computes the normalized color threshold for low stamina.
- Configures the slider's range from 0 to 100.
- Caches the slider fill image for color updates.
- Initializes the reflection adapter if the player controller is assigned.

Usage: Sets up the UI and ensures stamina data can be read safely.

2. Update

- Runs every frame to refresh the stamina display.
- Reads the current stamina percentage from the player controller via the adapter.
- Updates the slider value and changes its color based on thresholds:
 - Red for empty
 - Yellow for low
 - Green for normal
- Updates the text field with the current percentage if assigned.

Usage: Keeps the stamina UI responsive to the player's actions in real time.

◆ Helper Logic

1. IPlayerStaminaInfo Interface

- Defines a single property: the stamina percentage.
- Allows the monitor to interact with any player controller exposing this property.

2. Player Stamina Adapter

- Implements the interface and uses reflection to get the stamina value.
- Returns zero if the property is missing or invalid.
- Ensures flexibility: the monitor works with any compatible controller without hard dependencies.

⚡ Key Behavior

1. **Dynamic UI Updates** – The slider and optional text update in real time based on player stamina.
2. **Color Feedback** – Slider color changes provide immediate visual feedback on stamina status.

3. **Flexible Integration** – Works with any player controller exposing a StaminaPercent property.
4. **Normalized Display** – Converts raw stamina values into a 0–100 scale for consistent UI presentation.

◆ Summary

StaminaMonitor is a **versatile stamina UI manager** that:

- Reads stamina from any compatible controller via reflection
- Updates a slider and optional text in real time
- Provides visual feedback using color thresholds
- Can be customized at runtime through exposed properties

It is ideal for **player HUDs** in games where stamina management is important, giving clear and immediate feedback to the player.

Player Prefab Manager

✚ Overview

PlayerPrefabManager is an **editor utility for Unity** that provides menu commands to instantiate and configure specific player-related prefabs directly in the Unity hierarchy. It automates tasks that are normally manual, such as selecting the object, unpacking the prefab, and enabling rename mode. This is primarily a **workflow optimization tool for level designers or developers**.

⚙ Main Components

◆ Core Methods

1. Instantiate and Setup Prefab

- Responsible for locating a prefab in the project by name, instantiating it in the scene, and assigning a parent if specified.
- Steps performed:
 - Searches the project for prefab assets with the specified name.
 - Warns if multiple prefabs with the same name are found and uses the first one.
 - Converts the prefab asset reference into an actual GameObject in the scene.
 - Calls the "Complete Prefab Setup" method to finalize configuration.
- **Usage:** This is the internal method called by menu commands when creating a player prefab.

2. Complete Prefab Setup

- Takes an instantiated prefab and performs final adjustments:
 - Registers the object with Unity's undo system, allowing the creation to be undone.
 - Completely unpacks the prefab so it can be edited freely in the scene.
 - Selects the object in the hierarchy automatically.

- Triggers rename mode so the user can immediately rename the prefab.
- **Usage:** Ensures that newly instantiated prefabs are ready for immediate editing in the scene.

◆ Menu Items

- The script defines several menu commands accessible in Unity's "GameObject" menu under "Player Controller → 3D". Each menu item corresponds to a specific player prefab:
 1. **Base Player** – Instantiates the basic 3D player prefab.
 2. **Side View Player** – Instantiates the side-view 3D player prefab.
 3. **First Person Player** – Instantiates the first-person 3D player prefab.
 4. **Third Person Player Controller** – Instantiates the third-person 3D player prefab.
 5. **Third Person Camera Controller** – Instantiates the third-person camera controller prefab.
- Each menu item simply calls the "Instantiate and Setup Prefab" method with the prefab name and the currently selected object in the hierarchy as the parent.
- **Usage:** These menu items streamline scene setup, allowing developers to place player prefabs in the hierarchy with a single click and immediately start configuring them.

◆ Key Behavior

1. **Automatic Prefab Detection** – Uses the Unity asset database to find prefabs by name.
2. **Prefab Instantiation** – Automatically creates a copy of the prefab in the scene hierarchy.
3. **Prefab Unpacking** – Converts the prefab instance into a fully editable object.
4. **Automatic Selection** – The newly instantiated object is immediately selected in the hierarchy.
5. **Rename Mode Trigger** – Activates rename mode so the user can change the name without manually pressing F2.
6. **Undo Support** – Registers prefab creation with the undo system for easy reversals.

◆ Summary

PlayerPrefabManager is a **time-saving editor tool** designed to:

- Quickly place and configure player prefabs in a Unity scene.
- Handle automatic unpacking, selection, and renaming.
- Reduce repetitive manual steps for scene setup.
- Provide a clear and organized set of menu items for different player types.

It's primarily intended for developers and designers who frequently work with player prefabs in Unity and want to **streamline scene setup and prefab configuration**.

Scene Dropdown Manager Editor

✚ Overview

SceneDropDownManagerEditor is an **editor utility component for Unity** that provides a dropdown menu for selecting and loading scenes using a ScriptableObject, instead of relying on Build Settings. It works both in the Unity Editor's Edit Mode and Play Mode, allowing developers to quickly switch scenes during testing or development.

This tool is especially useful for **projects with many scenes**, where manually opening scenes through Build Settings would be cumbersome.

Main Components

◆ Serialized Fields

These are configurable references in the Unity Inspector:

1. Scene Dropdown

- A reference to a TextMeshPro dropdown component (TMP_Dropdown) that will list all available scenes.
- Usage: Displays the names of scenes stored in the ScriptableObject, allowing the user to select one.

2. Load Scene Button

- A reference to a UI button that triggers the loading of the currently selected scene in the dropdown.
- Usage: When clicked, the scene corresponding to the selected dropdown option is loaded.

3. Scene List

- A reference to a ScriptableObject containing a list of SceneAsset objects.
- Usage: Stores all the scenes available for selection. This allows the dropdown to populate dynamically without needing Build Settings.

◆ Private Fields

1. Scene Paths

- An internal list of strings storing the full paths of scenes derived from the ScriptableObject.
- Usage: Maps dropdown indices to actual scene paths for loading. Ensures the correct scene is loaded when the user clicks the button.

◆ Public Properties

- The component exposes public getters and setters for the dropdown, button, and SceneList.
- Usage: Allows other scripts or tools to programmatically assign or modify the UI elements or the scene list.

◆ Unity Lifecycle Methods

1. Start (Initialization)

- Ensures the ScriptableObject is assigned and contains scenes.
- Populates the dropdown with scene names.
- Registers the Load Scene button to trigger scene loading.
- Usage: Sets up the UI correctly when the component is enabled.

◆ Private Methods

1. Setup Dropdown

- Clears any existing dropdown options and internal scene paths.
- Converts each SceneAsset in the ScriptableObject to a readable scene name.
- Populates the dropdown with scene names.
- Detects the currently open scene and selects it by default in the dropdown.
- Usage: Dynamically fills the dropdown with scenes and ensures the correct selection is highlighted.

2. On Load Scene Button Clicked

- Validates the currently selected dropdown option.
- Retrieves the corresponding scene path from the internal list.
- Loads the selected scene:
 - In Play Mode: Uses an API that allows loading scenes during gameplay.
 - In Edit Mode: Opens the scene in the editor for editing.
- Usage: Allows the user to switch scenes immediately from the dropdown and button interface.

◆ Key Behavior

1. **Dynamic Scene Listing** – No reliance on Build Settings; scenes are listed based on the ScriptableObject.
2. **Editor & Play Mode Compatibility** – Correctly handles scene loading depending on whether Unity is in Edit Mode or Play Mode.
3. **Automatic Current Scene Selection** – The dropdown automatically highlights the currently open scene.
4. **Safe Loading** – Validates selected index and scene path to avoid errors.
5. **ScriptableObject Driven** – Makes it easy to manage scenes centrally, especially useful for large projects with frequent scene additions or removals.

◆ Usage Scenario

1. Assign the TMP_Dropdown and Button components in the Inspector.
2. Assign a ScriptableObject containing a list of SceneAsset objects.
3. When the game runs in the Editor or in Play Mode, the dropdown shows all scenes.
4. Select a scene from the dropdown and press the Load Scene button.
5. The selected scene is loaded immediately, either in the Editor or Play Mode.

This workflow simplifies **scene navigation for testing and development**, reduces manual scene opening, and avoids relying on Unity Build Settings.

Scene List Editor

Overview

SceneListEditor is a **ScriptableObject** used in the **Unity Editor** to store a list of scenes as SceneAsset objects. It is designed specifically for **editor-only scene management**, such as dropdown-based scene switching, without requiring scenes to be included in Build Settings.

It acts as a **centralized asset** that other editor tools can reference to populate UI elements like scene selectors.

Main Components

Serialized Fields

1. Scenes

- A list of SceneAsset objects.
- Usage: Each item in this list represents a scene that can be selected in editor tools (for example, in a dropdown menu).
- This list is the core of the ScriptableObject, providing a structured way to reference multiple scenes in one place.

ScriptableObject Features

1. CreateAssetMenu Attribute

- Allows this ScriptableObject to be created via the Unity Editor menu.
- Usage: Developers can create a new Scene List asset by navigating to **Player Controller** → **Debug** → **Scene List (Editor)**. This creates an asset that stores a list of scenes for editor tools.

Editor-Only Behavior

1. The scenes list is wrapped in a conditional compilation block, so it **only exists in the Editor**.
 - Usage: Ensures that this asset does not increase build size or affect runtime behavior in the final game.
 - This makes it purely an editor utility for development workflows.

How It Works with Other Tools

- The Scene List is typically referenced by scripts like the **Scene Dropdown Manager**:

1. The dropdown manager reads this list of scenes.
 2. Converts each SceneAsset into a scene name and path.
 3. Populates a dropdown menu in the Unity Editor with the available scenes.
 4. Allows the user to select and load a scene without relying on Build Settings.
- This provides a **flexible and dynamic way to manage scenes** during development.

◆ Usage Scenario

1. Create a new Scene List asset using the Unity Editor menu.
2. Add all the scenes you want to access via dropdowns or other editor tools to the scenes list.
3. Assign this asset to a scene management script like SceneDropdownManagerEditor.
4. The script can now display all the scenes from this list and allow quick switching between them directly in the Editor.

In summary, SceneListEditor is a **lightweight, editor-only container** for storing scene references, designed to work seamlessly with editor tools for scene management, improving workflow and scene accessibility during development.

Header Group Attribute

✿ Overview

This system provides a way to **group related fields in the Unity Inspector under collapsible headers**. It supports both **automatic usage via an attribute** and **manual usage in custom editors**.

The purpose is to improve **editor organization**, making large scripts easier to navigate by visually separating fields into meaningful sections.

⚙ Main Components

1. Attributes

- **HeaderGroupAttribute**
 - Marks a serialized field to indicate it belongs to a grouped section.
 - Must be used alongside Unity's standard [Header("...")].
- **UseHeaderGroupInspectorAttribute**
 - Marks a MonoBehaviour to enable the **automatic grouping system** in its custom inspector.
 - Only scripts with this attribute will have their fields grouped automatically.

2. HeaderGroupGUI (Manual API)

This is a static utility for manually drawing collapsible groups in **Custom Editors** or **Editor Windows**.

Key Variables:

- **foldoutStates**: Stores whether each group is currently open or closed.
 - Each group name maps to a Boolean value.
 - This ensures foldout states persist while editing.

Key Methods:

- **DrawGroup**
 - Draws a complete collapsible section.
 - Takes a group name and an action that draws the content inside the group.
 - Handles foldout rendering and indentation automatically.
- **BeginGroup / EndGroup**
 - Allows manual control when you want to start and end a collapsible section in multiple steps.
 - You can draw multiple fields inside the group manually.
- **Internal Utilities**:
 - Handles foldout drawing, background styling, and state saving.
 - Saves foldout open/close state in **EditorPrefs**, so groups remain open/closed across sessions.

3. HeaderGroupDrawer (Automatic Inspector)

This is a **custom inspector** that overrides the default Unity Inspector for MonoBehaviours marked with [UseHeaderGroupInspector].

Key Variables:

- **foldoutStates**: Tracks which groups are open or closed for the current script instance.

Key Methods:

- **OnInspectorGUI**
 - Iterates through all serialized fields of the MonoBehaviour.
 - Detects fields with [Header("...")] and [HeaderGroup].
 - Draws them inside collapsible sections using foldout logic.
 - Automatically handles starting and ending vertical sections and indentation.
- **DrawGroupBackground**
 - Draws the visual background behind the foldout header.
 - Applies semi-transparent shading for a clear separation.
 - Bold font style emphasizes the group title.
- **LoadFoldoutState / SaveFoldoutState**
 - Load and save each group's open/closed state to EditorPrefs.
 - Ensures the UI remembers which groups were expanded even after restarting the editor.

How It Works

1. Automatic Grouping

- Add [UseHeaderGroupInspector] to your MonoBehaviour.
- Mark fields with [Header("Group Name"), HeaderGroup].
- The custom inspector will automatically organize these fields into collapsible sections in the Unity Inspector.

2. Manual Grouping

- Use HeaderGroupGUI.DrawGroup("Group Name", action) inside a custom editor.
- Any fields or controls inside the action callback are drawn inside a collapsible section.
- Alternatively, use BeginGroup and EndGroup for finer control.

3. Persistence

- Open/close states of groups are saved in Unity's **EditorPrefs**.
- When reopening a script or Unity session, previously expanded/collapsed sections retain their state.

Usage Examples

Automatic:

```
[Header("Player Settings"), HeaderGroup]
```

```
public int health;
```

```
[Header("Player Settings"), HeaderGroup]
```

```
public int stamina;
```

Manual:

```
HeaderGroupGUI.DrawGroup("Stats", () =>
{
    EditorGUILayout.PropertyField(serializedObject.FindProperty("health"));
    EditorGUILayout.PropertyField(serializedObject.FindProperty("stamina"));
});
```

This provides flexibility for organizing inspector fields **both automatically and manually**, while maintaining a consistent look and foldout state.

In summary, this system is a **robust and flexible way to group related fields in the Unity Inspector**, improving readability and workflow for complex scripts.

It combines **attributes, editor GUI logic, and persistent state** to give a professional, collapsible section experience.

Read Only In Inspector Attribute

Overview

This system provides a way to **display fields in the Unity Inspector as read-only**, meaning the values can be viewed but not edited directly in the Inspector. It is especially useful for:

- Showing **runtime values**.
- Displaying **debug information**.
- Preventing accidental modifications while still allowing code to update the field.

Main Components

1. ReadOnlyInInspectorAttribute

- **Purpose:** Marks a field so that the Inspector will render it as disabled (read-only).
- **Behavior:**
 - The field cannot be edited manually in the Inspector.
 - The field **can still be modified through code** at runtime or in scripts.
- **Usage:** Attach it alongside [SerializeField] on private fields, or on public fields, like this:

```
[SerializeField, ReadOnlyInInspector]
```

```
private int health;
```

This allows the value to be serialized and visible in the Inspector but not editable.

2. ReadOnlyInInspectorDrawer (Property Drawer)

- **Purpose:** Handles the rendering logic for all fields marked with ReadOnlyInInspectorAttribute.
- **Key Logic:**

1. **Disables the GUI:** Temporarily prevents user interaction with the field in the Inspector.
 2. **Draws the field normally:** Ensures the field value is displayed, just like a regular Inspector field.
 3. **Re-enables the GUI:** Restores the ability to edit other fields in the Inspector.
- **Effect:** From the user's perspective in the Inspector, the field appears grayed out and cannot be changed, but still shows its current value.

How It Works

1. You apply the `ReadOnlyInInspector` attribute to a field you want to display as read-only.
2. When Unity renders the Inspector, it detects this attribute and delegates the drawing to the `ReadOnlyInInspectorDrawer`.
3. The drawer disables editing temporarily, draws the field, and restores editing afterward.
4. Any code that updates the field will still reflect in the Inspector in real-time.

Usage Example

- **Debugging Runtime Values:** Show the player's current health or stamina during gameplay without letting designers accidentally modify it.
- **Monitoring Calculated Properties:** Display a calculated score or AI state that is updated in code.

Example in Inspector:

- A private health field:

```
[SerializeField, ReadOnlyInInspector]  
private int health;
```

- In Play Mode, Unity will show the health value in gray, and it will update as the player takes damage, but the designer cannot type a new value manually.

Summary

- **Variables:**
 - The field you mark with the attribute (any serializable field).
- **Methods:**
 - The property drawer temporarily disables editing, draws the field, then restores normal Inspector functionality.
- **Usage:**
 - Ideal for displaying **read-only data** in the Inspector.

- Helps prevent accidental edits while still providing visibility of runtime or debug information.

This is a **lightweight, editor-only enhancement** that improves usability and clarity in Unity's Inspector without affecting runtime behavior.

Tags Dropdown Attribute

Overview

This system provides a **dropdown menu in the Unity Inspector** for string fields that are intended to reference existing Unity tags. Instead of manually typing a tag, the user can select from the list of tags defined in the project. This helps prevent typos and ensures the string value always matches a valid tag.

Main Components

1. TagsDropdownAttribute

- **Purpose:** Marks a string field so that it will display a dropdown of Unity tags in the Inspector.
- **Behavior:** Does not change the field type or how it's serialized; it only affects how the field is drawn in the Editor.
- **Usage:** Apply it to string fields that are meant to store tag names. For example:

```
[SerializeField, TagsDropdown]
```

```
private string enemyTag;
```

- **Effect:** The field will render as a dropdown with all available Unity tags.

2. TagsDropdownDrawer (Property Drawer)

- **Purpose:** Handles the visual rendering and interaction logic for fields marked with TagsDropdownAttribute.
- **Key Behavior:**
 1. **Type Checking:** Ensures that the field is a string. If it is not, it displays a warning label.
 2. **Fetches Tags:** Retrieves all the tags defined in the Unity project.
 3. **Handles Missing Tags:** If the current string value does not match any existing tag, it adds a placeholder entry labeled "Tag Missing (current value)".
 4. **Displays Dropdown:** Shows a popup list containing all tags, plus the "Tag Missing" option if needed.
 5. **Updates Value:** When the user selects a valid tag from the dropdown, the string value of the field is updated accordingly.

6. **Warning Display:** If the selected string is invalid or missing from the project's tags, a warning message appears below the field in the Inspector.
7. **Height Calculation:** Dynamically adjusts the field's height in the Inspector to account for the extra warning box if necessary.

How It Works

1. You apply the TagsDropdown attribute to a string field that represents a tag.
2. Unity detects this attribute in the Editor and delegates drawing to the TagsDropdownDrawer.
3. The drawer retrieves the list of all tags in the project.
4. If the current string value does not match a valid tag, it adds a "Tag Missing" entry to the dropdown and displays a warning.
5. The dropdown is rendered in the Inspector. Selecting a valid tag updates the field; selecting the missing placeholder shows a warning.
6. The drawer calculates the correct vertical space, so the dropdown and any warning message fit properly in the Inspector.

Usage Examples

- **Tag Reference for Enemies:** Prevents typos when assigning an enemy tag to an AI or trigger system.
- **Layered Systems:** Any system that relies on tags, like collision detection or event filtering, can benefit from this dropdown.
- **Inspector Clarity:** Designers can see all possible tags and ensure consistency across prefabs and scenes.

Example in Inspector:

- Field with TagsDropdown:
- Enemy Tag: [Dropdown: "Enemy", "Player", "Collectible"]
- If a previously saved value does not match any tag:
 - Dropdown shows "Tag Missing (OldValue)"
 - A small warning appears: "String value does not match any tag!"

Summary

- **Variables:**
 - Field marked with TagsDropdownAttribute.
 - Internal list of Unity tags.
 - Current selected tag and a placeholder for missing tags.
- **Methods:**
 - Rendering the dropdown in the Inspector.
 - Updating the string field when a valid tag is selected.
 - Displaying warnings for invalid tag values.
 - Calculating the necessary height for proper layout.
- **Usage:**

- Simplifies assigning tags.
- Prevents errors from typos.
- Improves workflow for designers and developers by showing all project tags in a clear, selectable dropdown.

This is an **editor-only enhancement** that improves Inspector usability without affecting runtime behavior.

Validate Reference Attribute

Overview

This system provides a way to **highlight missing object references** in the Unity Inspector. It is useful for ensuring that fields which are meant to reference objects (like prefabs, scripts, or assets) are not left empty. If a reference is missing, the field is visually highlighted, and a message is displayed to alert the developer.

Main Components

1. ValidateReferenceAttribute

- **Purpose:** Marks a field in a Unity component so that the Inspector checks whether the reference is assigned.
- **Behavior:**
 - When applied to an object reference field, it triggers the custom drawer to perform validation.
 - Can be configured to show missing references either as an **error** (red highlight) or a **warning** (yellow highlight).
- **Variables:**
 - UseError: A boolean flag indicating if a missing reference should be treated as an error (true) or as a warning (false).
- **Usage Examples:**
 - [ValidateReference]: Missing reference is treated as an error by default.
 - [ValidateReference(false)]: Missing reference is treated as a warning.
- **Effect:** Improves code safety and reduces runtime null-reference bugs by visually reminding the developer to assign necessary references.

2. ValidateReferenceDrawer (Property Drawer)

- **Purpose:** Handles the visual rendering and validation logic for fields marked with ValidateReferenceAttribute in the Inspector.
- **Key Behavior:**
 1. **Check Field Type:**
 - Validates that the property is an object reference. If the field is not a reference type, it ignores it.
 2. **Check for Missing Reference:**
 - Determines if the object reference is empty (null).
 3. **Highlight Field:**
 - If empty, the field is highlighted with:

- **Red** for error mode.
- **Yellow** for warning mode.
- 4. **Draw Property Field:**
 - Displays the normal Inspector field, but with the highlight color applied.
- 5. **Display Help Message:**
 - If the field is empty, a message box is shown below the field.
 - The message indicates what type of object is expected (e.g., "Put an item of type GameObject here!").
 - The message type matches the chosen mode: error or warning.
- 6. **Calculate Field Height:**
 - The drawer dynamically increases the vertical space for the property when the help message is shown to ensure proper layout in the Inspector.

How It Works

1. You apply the ValidateReference attribute to any object reference field that should not be empty.
2. Unity delegates the rendering of this field to the ValidateReferenceDrawer in the Editor.
3. The drawer checks whether the field is null:
 - If it has a value, it is displayed normally.
 - If it is empty:
 - The field background is colored (red or yellow depending on error mode).
 - A help message appears below the field indicating the required type.
4. The drawer automatically calculates the required height to accommodate both the field and the message.
5. Developers see a clear, visual cue in the Inspector, helping them prevent unassigned references.

Usage Examples

- **Required Prefabs:** Ensures that a prefab field for a player or enemy is never left unassigned.
- **Script References:** Prevents accidentally leaving component references null.
- **Assets:** Highlights missing texture, audio clip, or material references in a component.

Example in Inspector:

- Field: PlayerController
 - Assigned → normal display.
 - Missing → highlighted in red (or yellow), with message: "Put an item of type 'PlayerController' here!"

Summary

- **Variables:**
 - UseError: Determines whether missing references are treated as errors or warnings.

- Field reference in the component being validated.
- **Methods:**
 - **Draw GUI:** Renders the field, applies highlight, and displays the help message if empty.
 - **Get Property Height:** Adjusts vertical space for the help message.
- **Usage:**
 - Apply [ValidateReference] to any field that must not be empty.
 - Provides immediate visual feedback in the Unity Inspector.
 - Improves safety and reduces runtime null-reference errors.

This system is **editor-only** and does not affect runtime behavior but greatly improves workflow reliability and debugging efficiency.

Third Person Camera Controller

Overview

This script implements a **third-person camera system** that follows a player, allows rotation, supports zooming, and dynamically handles collisions with the environment. It works with Unity's modern Input System and provides smooth, customizable camera behavior for third-person games.

Key Concepts

The camera controller uses the following key ideas:

1. **Rotation:** The camera orbits around a pivot point attached to the player, controlled by player input.
2. **Zoom:** The camera can move closer or farther from the player, with configurable minimum and maximum distances.
3. **Collision Handling:** The camera adjusts its distance dynamically if obstacles appear between the player and the camera to avoid clipping.
4. **Input System Integration:** Supports zooming and rotation through Unity's Input System.
5. **Customization:** Provides adjustable sensitivity, vertical inversion, pitch limits, and smoothing for collisions.

Variables

Input Settings

- **Zoom Action:** References the input action used to zoom the camera in or out.
- **Look Input Handler:** Captures input for rotating the camera around the player.

References

- **Player Transform:** The main object representing the player.
- **Player Pivot Transform:** The point around which the camera rotates.
- **Main Camera Pivot Transform:** The pivot used to orient the camera based on rotation.

- **Collision Camera Pivot Transform:** The transform that moves to prevent the camera from clipping into objects.

Camera Settings

- **Camera Sensitivity:** Global multiplier for rotation speed.
- **Angle Sensitivity:** Separate multipliers for horizontal (yaw) and vertical (pitch) rotation.
- **Min/Max Pitch:** Limits vertical rotation to prevent the camera from flipping or going under/over the player.
- **Invert Y:** Option to invert vertical camera movement.

Distance Settings

- **Min/Max Distance:** Limits how close or far the camera can zoom.
- **Min Collision Distance:** Minimum distance the camera can reach when avoiding obstacles.
- **Zoom Speed:** Controls how fast the camera zooms.
- **Collision Smooth Speed:** Controls how smoothly the camera reacts to collisions.
- **Target Camera Distance:** Desired distance from the pivot.

Collision Settings

- **Collision Detection Layers:** Layers considered when checking for obstacles.
- **Collision Offset:** Small offset to prevent clipping into surfaces.

Private/Internal Fields

- **Current Pitch and Yaw Angles:** Track the current rotation of the camera.
- **Current Camera Distance:** Tracks the current actual distance from the pivot, factoring in collisions.
- **Raycast Buffer:** Reusable array for collision raycasting to minimize memory allocations.
- **Camera Active Flag:** Determines whether the camera logic is enabled.

◆ Methods

Initialization & Lifecycle

- **Awake:** Validates that all necessary references are assigned and that configuration values are consistent (e.g., min pitch < max pitch).
- **Start:** Initializes camera distance, clamps it, binds zoom input, and performs the first collision check.
- **OnDestroy:** Cleans up input bindings to avoid memory leaks.
- **Update:** Called every frame to handle camera rotation and collision adjustments.
- **OnDrawGizmosSelected:** Draws visual debug markers for the camera pivots and lines in the editor.

Camera Rotation

- **Process Look Input:**
 - Moves the camera pivot to the player's position.
 - Reads input from the look handler and applies sensitivity.

- Updates yaw (horizontal) and pitch (vertical) angles.
- Clamps pitch to prevent flipping.
- Applies the final rotation to the main camera pivot.

Camera Collision

- **Process Camera Collision:**
 - Calculates the desired camera position based on rotation and zoom.
 - Performs a non-allocating raycast toward the desired position to detect obstacles.
 - Ignores collisions with the player or child objects.
 - Determines the closest obstacle and adjusts the camera distance accordingly.
 - Smoothly interpolates to the corrected distance.
 - Updates the collision camera pivot's local position to reflect the adjusted distance.

Public API

- **Set Camera Sensitivity:** Adjusts global rotation speed.
- **Set Invert Y-Axis:** Enables or disables vertical inversion.
- **Set Camera Active:** Enables or disables camera processing entirely.

◆ Usage

1. **Setup in Scene:**
 - Attach this script to a camera GameObject or empty GameObject that represents the camera rig.
 - Assign the player's root and pivot transforms.
 - Assign the main and collision camera pivots.
 - Configure the input actions for rotation and zoom.
2. **Adjust Camera Settings:**
 - Set desired sensitivity, min/max pitch, and invert Y if preferred.
 - Configure zoom distance limits and speeds.
 - Set collision layers and offsets according to your environment.
3. **Play:**
 - The camera follows the player, smoothly rotates based on input, and avoids clipping into objects dynamically.
 - Zooming is responsive to input.
 - Collisions automatically adjust the camera distance without abrupt changes.

✅ Summary

This controller provides a **robust, flexible third-person camera** with:

- Smooth rotation and zoom.
- Collision handling to avoid clipping into objects.
- Input System integration for modern input handling.
- Easy customization through exposed settings.
- Editor-friendly visual debugging for camera pivots.

It is intended for **3D player characters** where a dynamic, smooth third-person perspective is required.

Third Person Player Controller 3D

Purpose of the Script

This is a modular 3D player controller designed for a third-person game. It handles:

- Player input for movement, running, jumping, and crouching.
- Physics-based movement and collision detection.
- Stamina management for running.
- Camera adjustments depending on player state.
- Modular and extensible structure using sensors and controllers.

It is structured so that each part of the system (movement, jumping, rotation, stamina, collision detection) is separated for clarity, modularity, and easier maintenance.

Main Categories of Variables

1. Input Variables

- Movement, run, jump, and crouch actions.
- These define which buttons or keys the player uses to control the character.

2. References

- Player's position and orientation in the world (transform).
- Physics body that reacts to forces (rigidbody).
- Colliders for standing and crouching, to adjust collision size.
- Camera pivots for local and world-space camera movement.

3. Camera Settings

- Positions for standing and crouching camera views.
- Speed for smooth interpolation between camera positions.

4. Movement Settings

- Walk speed, run speed, crouch walk/run speed.
- Jump force and maximum jump count.
- Smooth factor for rotation.
- "Coyote time" allowing jump shortly after leaving the ground.

5. Stamina Settings

- Maximum stamina, minimum stamina required to run.
- Rate of stamina depletion when running and recovery when idle.

6. Player State Settings

- Control mode (automatic or manual).
- Current player ability (jump, crouch, or none).
- Movement mode (just walking or running allowed).

7. Collision Layers

- Define which layers are considered ground, obstacles, or ceilings for sensor detection.

8. Sensor Settings

- Sizes and positions for ground, obstacle, and ceiling sensors.
- Lists of tags to ignore during collision detection.

9. Physics Materials

- Materials with high or low friction for the ground.
- Maximum slope angle that can be walked on.

10. Debug Settings

- Booleans to enable visual debugging of sensors and slope angles.

11. Runtime Fields

- Sensors for ground, obstacle, ceiling, and slope validation.
- Controllers for jumping, movement, rotation, and stamina.
- Input event configurations for reacting to player input.
- Current state variables: grounded, crouching, running, speed, and playable.
- Cached transform data and obstacle/ceiling sensor positions.
- Timer to prevent rapid toggling of crouch.

Core Methods

1. Initialization

- **ValidateSerializedReferences**: Ensures that all required references (colliders, rigidbody, camera pivot) are assigned and logs warnings if anything is missing.
- **InitializeComponents**: Sets initial states and calls setup methods for sensors, controllers, and input events.
- **SetupSensors**: Instantiates collision sensors for ground, obstacles, ceiling, and slope validation.
- **SetupControllers**: Creates separate controllers for jump, movement, rotation, and stamina.
- **SetupInputBindings**: Maps input events to actions (jumping, moving, crouching, running).

2. Frame Updates

- **CacheTransformData**: Stores the current player position and rotation for sensor calculations.
- **UpdateSensorPositions**: Adjusts sensor positions dynamically based on crouching state.
- **UpdateGroundedState**: Determines if the player is on the ground and if crouch input should be allowed after landing.
- **UpdateAbilityAndCrouchLogic**: Handles ability changes, crouch toggling, and updates if running is allowed.
- **UpdateSpeedAndColliders**: Sets movement speed depending on running/crouching and switches colliders accordingly.
- **UpdateCameraPivotPosition**: Smoothly interpolates the camera between standing and crouching positions.

3. Input Handlers

- Jumping triggers jump controller with force and manages crouch blocking.
- Movement updates movement controller with directional input.
- Crouch toggles crouch state depending on control mode and sensors.
- Running activates stamina-based speed control and limits running if stamina is low.

4. Public API

- Change control mode, player abilities, running mode, or toggle whether the player can be controlled.
- Save and load the player's state (position, rotation, abilities, and other flags) as a serialized dictionary for persistence.

5. Cleanup

- Disposes all sensors, controllers, and unbinds input events when the player is destroyed or scene unloaded.

6. Debug Gizmos

- Draws visual markers for camera positions (standing and crouching) in the editor.
- Connects the two camera positions with a line for easier visualization.

Usage

- **Setup:** Attach the controller to a player object. Assign references to the rigidbody, colliders, camera pivots, and input actions.
- **Gameplay:** The script will handle movement, running, jumping, crouching, stamina depletion/recovery, collision detection, and camera adjustments automatically.
- **Extensibility:** The modular design allows you to swap or extend controllers (e.g., add a dash ability or new sensors) without rewriting the core logic.
- **Persistence:** The player's state can be saved and loaded at any time, making it compatible with save systems.

In summary, this script provides a full-featured, modular third-person player controller that handles physics, input, stamina, and camera logic while allowing easy extension and robust debugging. It separates responsibilities cleanly using sensors and controllers so each gameplay system works independently but cohesively.

First Person Camera Controller

This script is a **first-person camera controller** designed for a 3D game environment. Its main function is to rotate the player character horizontally and the camera vertically based on input, typically from a mouse or controller.

Serialized Fields (Configurable in Editor)

These are parameters and references you can set in the Unity editor:

1. **lookInputHandler** – This is an object responsible for reading camera input (like mouse or joystick movement). It provides a direction vector that the script uses to determine how the camera and player rotate.
2. **playerTransform** – The part of the player that rotates horizontally. This is usually the character's body or root object, rotated around the vertical axis.
3. **cameraPivot** – The part of the camera rig that rotates vertically (up and down). The player body does not tilt; only this pivot does.
4. **cameraSensitivity** – A global multiplier for how fast the camera reacts to input. Lower values = slower response, higher = faster.
5. **angleSensitivity** – Separate multipliers for horizontal (yaw) and vertical (pitch) rotation. This allows finer control per axis.
6. **minPitch** – The lowest angle the camera can look downwards.
7. **maxPitch** – The highest angle the camera can look upwards.
8. **invertY** – If true, vertical input is inverted (pushing mouse up looks down).

Runtime Fields (Internal State)

- **currentPitch** – Tracks the camera's vertical rotation. This is used to ensure the camera doesn't exceed the min/max pitch limits.

Properties

These expose the serialized fields for other scripts to read or modify, e.g., adjusting sensitivity or enabling Y-axis inversion during gameplay.

Unity Lifecycle Methods

1. **Awake()**
 - Ensures all required references (input handler, player, and camera pivot) are assigned.
 - Checks that the minimum pitch isn't larger than the maximum pitch, which would cause incorrect behavior.
2. **Update()**
 - Called every frame.
 - Delegates camera rotation handling to the `HandleMouseLook()` method.

Camera Control Methods

1. **HandleMouseLook()**
 - Reads the input from `lookInputHandler`.
 - Applies sensitivity multipliers.
 - **Yaw (horizontal rotation):**
 - Applied to the player object.
 - Rotates around the vertical axis to turn left/right.
 - **Pitch (vertical rotation):**
 - Applied to the camera pivot.
 - Updates `currentPitch` with input, clamps it between `minPitch` and `maxPitch`.
 - Prevents the camera from looking too far up or down.
2. **SetSensitivity(float)**
 - Changes the global sensitivity of the camera.
 - Clamps the value between 0.05 and 1.0 to avoid extreme speeds.
3. **SetInvertY(bool)**
 - Enables or disables vertical input inversion.
 - When true, moving the mouse up looks down, and vice versa.

How It Works in Practice

- The player moves the mouse or joystick.
- The `lookInputHandler` provides a directional input vector.
- `HandleMouseLook()` reads this input:

- Converts horizontal input to yaw and rotates the player.
 - Converts vertical input to pitch and rotates the camera pivot, respecting min/max limits and optional inversion.
- This allows smooth first-person camera movement that follows the player's input naturally.
- Other scripts can adjust sensitivity or inversion on the fly using the public methods.

Summary of Usage

- Attach this script to a player object.
- Assign the player body (for yaw rotation) and camera pivot (for pitch rotation) in the editor.
- Link an input handler that provides look direction (mouse, joystick, or controller).
- Configure sensitivity, min/max pitch, and optional Y-axis inversion.
- During gameplay, the player can look around freely within the defined pitch limits.

First Person Player Controller 3D

This script is a complete **3D first-person player controller** for Unity, designed for modularity, flexibility, and extensibility. It handles movement, jumping, crouching, running, stamina, collision detection, and camera control.

Overview

The controller is built around the concept of **modular subsystems**:

1. **Input Handling** – receives player input for movement, running, jumping, and crouching.
2. **Collision Sensors** – detect the ground, obstacles, and ceilings.
3. **Movement Controllers** – manage movement, jumps, and stamina.
4. **Camera Logic** – adjusts camera position smoothly according to crouching or standing.
5. **State Management** – keeps track of abilities, movement mode, grounded state, and crouching.

It supports two control modes:

- **Automatic**: the script manages player abilities automatically.
- **Manual**: player input overrides some automatic behaviors.

Key Variables and What They Do

Input Settings

- **Movement, Run, Jump, Crouch Inputs**: Capture the player's actions and map them to events. These define how the player moves, starts running, jumps, or crouches.

Player References

- **Player Transform:** Represents the position and orientation of the player in the world.
- **Player Rigidbody:** Physics-based body that handles movement and collisions.
- **Standing Collider:** Collider shape for standing.
- **Crouching Collider:** Collider shape for crouching.

Camera Settings

- **Camera Pivot:** Transform controlling the camera's rotation and vertical position.
- **Standing/Crouching Local Positions:** Defines where the camera is relative to the player in each stance.
- **Camera Lerp Speed:** Controls how smoothly the camera moves between standing and crouching positions.

Movement Settings

- **Walking, Running, Crouch Walk/Run Speeds:** Base speeds for various player states.
- **Jump Force:** Upward force applied when jumping.
- **Max Jump Count:** Allows double jump or multiple jumps.
- **Coyote Time:** Small time buffer allowing jumps even just after leaving the ground.

Stamina Settings

- **Max Stamina:** Maximum stamina value.
- **Min Stamina for Running:** Minimum threshold required to run.
- **Stamina Depletion/Recovery Rates:** Controls how quickly stamina drains when running and regenerates when idle.

Player State

- **Control Mode:** Automatic or manual input behavior.
- **Current Ability:** The skill or action player can perform (e.g., crouch, jump).
- **Movement Mode:** Walk only or run enabled.

Collision and Sensor Settings

- **Ground Layers / Obstacle Layers / Ceiling Layers:** Define what surfaces are considered ground, obstacles, or ceilings.
- **Ground/Obstacle/Ceiling Sensors:** Box-shaped areas that detect collisions to determine grounded state, obstacles, or ceilings.
- **Slope Angle Sensors:** Check if the current surface slope is walkable.
- **Physics Materials:** High and low friction materials for different surface interactions.

Runtime Fields

- **Collision Sensors:** Track if player is grounded, obstructed, or blocked by a ceiling.
- **Controllers:** Handle jumping, movement, and stamina logic.
- **Input Events:** Map input actions to player behaviors.
- **Movement & State Flags:** Indicate if the player is grounded, crouching, running, playable, etc.
- **Timing Flags:** Prevent rapid toggling of crouch, and control input timing.

Core Methods and What They Do

Initialization Methods

- **Awake:** Checks that all necessary references are assigned. Logs warnings if missing.
- **Start:** Prepares runtime objects, initializes sensors, controllers, and binds input events.
- **Setup Sensors:** Configures collision detection for ground, obstacles, and ceiling.
- **Setup Controllers:** Instantiates logic for jump, movement, and stamina.
- **Setup Input Bindings:** Connects input actions (movement, run, jump, crouch) to behavior methods.

Update Methods

Executed every frame:

1. **CacheTransformData:** Updates the player's current position and rotation for sensors.
2. **UpdateSensorPositions:** Moves sensors based on the player's movement and crouching state.
3. **UpdateGroundedState:** Checks if the player is touching the ground, including slope checks and crouch allowances.
4. **UpdateAbilityAndCrouchLogic:** Handles transitions between abilities and crouching states.
5. **UpdateSpeedAndColliders:** Adjusts movement speed based on running, walking, crouching, and stamina. Switches the active collider for crouch vs. standing.
6. **UpdateCameraPivotPosition:** Smoothly moves the camera according to the player's stance.

Public API Methods

- **SetControlMode:** Switches between automatic and manual modes.
- **SetPlayerAbility:** Changes the active ability, e.g., enabling/disabling crouching or jumping.
- **SetRunEnabled:** Enables or disables running.
- **TogglePlayable:** Determines whether the player can respond to input.
- **SavePlayerData / LoadPlayerData:** Serializes or restores player state including position, rotation, abilities, and crouch state.

Cleanup Methods

- **DisposeAll:** Frees all sensors, controllers, and input bindings to avoid memory leaks when the object is destroyed.

Usage

1. Attach this script to a player object in Unity.
2. Assign references in the editor:
 - Player transform and rigidbody.
 - Colliders for standing and crouching.
 - Camera pivot.

- Input actions (move, jump, crouch, run).
- Physics materials and sensor settings.
- 3. Adjust movement, jump, stamina, and camera parameters for desired behavior.
- 4. At runtime, the script automatically handles:
 - Input mapping to movement.
 - Running and stamina depletion.
 - Jumping with coyote time support.
 - Crouching, including obstacle detection to prevent standing.
 - Collision detection for ground and slope checks.
 - Smooth camera positioning for standing and crouching.

This controller is **extensible**, allowing developers to add new abilities, modify sensors, or change input logic without modifying core movement mechanics.

Side View Player Controller 3D

Overview

This script is a complete system for controlling a 3D player character in a side-view Unity game. It manages:

- Player input (moving, running, jumping, crouching)
- Physical movement and collisions
- Rotation smoothing
- Stamina management
- Sensors to detect ground, obstacles, and ceilings
- Gameplay state handling (abilities, movement mode, and playability)
- Saving and loading player state

The controller is modular, meaning that movement, rotation, stamina, and jump mechanics are handled by separate subsystems that interact with sensors and input events.

Core Concepts

1. Player Abilities and Modes

- **Control Modes:** Automatic (normal gameplay) or Manual (player cannot crouch automatically).
- **Abilities:** Determines what the player can do (jump, crouch, none).
- **Movement Modes:** Either walk-only or walk-and-run.

2. Sensors

Sensors are invisible boxes around the player that detect collisions and environmental conditions:

- **Ground Sensor:** Checks if the player is standing on a valid surface.
- **Obstacle Sensor:** Detects obstacles in front to prevent moving through walls.
- **Ceiling Sensor:** Detects obstacles above the player to prevent standing up while crouched.
- **Slope Sensor:** Validates if the ground angle allows walking.

3. Player Controllers

These components manage specific aspects of player movement:

- **Jump Controller:** Applies vertical force and manages double-jump logic.
- **Movement Controller:** Moves the player horizontally and prevents collisions with obstacles.
- **Rotation Controller:** Smoothly rotates the player toward the movement direction.
- **Stamina Controller:** Controls running speed and stamina depletion/recovery.

4. Input Events

Input is mapped to actions like moving, jumping, crouching, and running. Events handle:

- Pressed (trigger once)
- Released (trigger once when released)
- Held (continuous trigger while button is pressed)

5. State Tracking

The script keeps track of:

- Whether the player is grounded
- Whether the player is crouching
- Whether the player is running
- Whether the player is allowed to move (playable)
- Current input direction
- Current movement speed
- Timing for crouch toggles to prevent rapid switching

6. Stamina and Speed

- The player has a stamina pool that depletes while running.
- If stamina is too low, running is disabled.
- Speed changes depending on whether the player is walking, running, or crouched.

Key Variables

- **Input References:** Stores input mappings for movement, run, jump, and crouch.
- **Player Components:** Transform, physics body, colliders.
- **Movement Values:** Speeds for walking, running, crouching, and rotation smoothing.
- **Jump Values:** Jump force, double-jump limit, coyote time (short grace period to jump after leaving ground).
- **Stamina:** Maximum, minimum for running, depletion, and recovery rates.
- **Collision Layers:** Determines which layers are treated as ground, obstacles, or ceilings.
- **Sensor Sizes and Centers:** Defines positions and sizes of collision boxes.
- **Physics Materials:** High and low friction materials to manage how the player interacts with slopes.
- **Debug Flags:** Toggles visualization of sensors and slope angles.

Core Methods

1. Awake / Start

- Validates that all necessary references (colliders, rigidbody, input actions) are set.
- Initializes sensors, controllers, and input bindings.

2. Update (called every frame)

- Updates the player's position and rotation for sensor checks.
- Adjusts sensor positions and sizes based on crouching.
- Checks if the player is grounded using sensors and slope angle.
- Updates abilities and crouching logic depending on control mode.
- Updates movement speed and activates the correct collider.
- Updates crouch cooldown timer to avoid rapid toggling.

3. Setup Methods

- **SetupSensors**: Creates the collision sensors and slope angle checkers.
- **SetupControllers**: Initializes movement, rotation, jump, and stamina controllers.
- **SetupInputBindings**: Maps player actions to input events (jump, move, crouch, run).

4. Player State Control

- **SetControlMode**: Switches between automatic and manual input control.
- **SetPlayerAbility**: Changes what abilities the player can currently use.
- **SetRunEnabled**: Enables or disables running ability.
- **TogglePlayable**: Temporarily disables player input.

5. Saving and Loading

- Converts player position, rotation, state, abilities, and movement mode into a JSON string for saving.
- Loads saved state from JSON and updates player's transform and status.

6. Cleanup / Dispose

- Removes all sensors, controllers, and input events to free memory when the object is destroyed.

Usage

1. Attach the script to a player object in a 3D Unity scene.
2. Assign:
 - Rigidbody for physics-based movement
 - Colliders for standing and crouching
 - Input actions for movement, jump, run, and crouch
 - Optional: high and low friction physics materials
3. Set movement speeds, jump forces, stamina, and sensor sizes according to the desired gameplay.
4. Control the player:
 - Automatic mode handles crouching and jumping automatically.
 - Manual mode requires explicit input to crouch.
 - Stamina limits running and influences movement speed.
 - Sensors prevent walking through walls or standing up under obstacles.
5. Can save and load player state to maintain position, abilities, and mode between sessions.

In short, this script is a **modular, highly configurable player controller** designed for side-view 3D games. It combines sensors, input events, and dedicated controllers to manage all aspects of movement, collision detection, rotation, and stamina, while providing an API for saving/loading and gameplay control modes.

Purpose

This script provides a modular 3D player controller for a game engine. It handles everything related to player movement, including:

- Walking and running
- Jumping (including double jumps)
- Crouching
- Rotation and smooth turning
- Collision detection with ground, obstacles, and ceilings
- Stamina management
- Saving and loading player state

The system is designed to be flexible and extendable, with sensors, controllers, and input events separated into modular parts.

Core Concepts and Variables

1. Player Control Modes

- Determines how the player input is processed:
 - **Automatic:** The controller manages actions like crouching automatically.
 - **Manual:** The player has full manual control; some actions may be restricted.

2. Player Abilities

- Defines what the player can do:
 - No abilities
 - Crouch
 - Jump

3. Movement Modes

- Determines if the player can only walk or run.

Input Actions

- **Movement:** Directional input from the player.
- **Run:** Trigger to start running.
- **Jump:** Trigger to jump.
- **Crouch:** Trigger to crouch or stand up.

References

- **Player Transform:** Position and rotation of the player in the world.
- **Player Rigidbody:** Physics component used to move the player.
- **Colliders:** Shapes representing the player's body when standing or crouching. Only one is active at a time.
- **Physics Materials:** Materials that affect friction on surfaces.

Movement Settings

- **Walk and Run Speeds:** Base speeds for different movement modes.
- **Crouch Speeds:** Adjusted speed while crouching.
- **Rotation Smoothness:** How smoothly the player turns.
- **Jump Force and Count:** Strength and maximum consecutive jumps allowed.
- **Coyote Time:** Small grace period to allow jumping just after leaving a platform.
- **Global Input Rotation Offset:** Angle adjustment for input direction.

Stamina Settings

- **Maximum Stamina:** Maximum energy the player can have.
- **Minimum Stamina to Run:** Threshold before running is disabled.
- **Depletion and Recovery Rates:** How quickly stamina decreases when running and recovers when idle.

Sensors

Sensors are modular checks for collisions and environmental conditions:

1. **Ground Sensor:** Checks if the player is touching the ground.
2. **Obstacle Sensor:** Checks for walls or objects in front of the player.
3. **Ceiling Sensor:** Prevents standing up when there's an overhead obstruction.
4. **Slope Angle Sensor:** Ensures the player can only walk on slopes below a maximum angle.

Each sensor has parameters for position, size, rotation, layers to detect, ignored tags, and debug visualization.

Controllers

Controllers handle the actual mechanics of movement:

- **Jump Controller:** Applies vertical forces for jumping and handles multiple jumps and coyote time.
- **Move Controller:** Applies horizontal movement, checking for obstacles.
- **Rotation Controller:** Smoothly rotates the player toward movement direction.
- **Stamina Controller:** Modifies speed based on stamina, handles depletion and recovery.

Runtime State Variables

- **Position and Rotation Cache:** Stores the player's current location and orientation.
- **Movement Input:** Current directional input vector.

- **Current Speed:** Final computed speed depending on running, crouching, and stamina.
- **Flags:** Grounded, crouching, running, playable, and ability to run.
- **Crouch Timer:** Prevents rapid toggling of crouch state.

Initialization Process

1. **Validation:** Ensures required references like input actions, colliders, Rigidbody, and physics materials are assigned.
2. **Sensors Setup:** Instantiates all collision sensors and slope angle checks.
3. **Controllers Setup:** Initializes movement, jump, rotation, and stamina controllers.
4. **Input Bindings:** Connects input actions (jump, move, crouch, run) to respective controllers.

Update Loop

Called every frame:

1. **Cache Transform Data:** Updates position and rotation for sensors.
2. **Update Sensor Positions:** Adjusts sensors based on crouch or normal state.
3. **Check Grounded State:** Uses ground sensor and slope validation.
4. **Ability & Crouch Logic:** Handles control mode rules, crouching, and running permissions.
5. **Update Speed and Colliders:** Sets movement speed and enables/disables standing or crouching colliders.
6. **Crouch Timer Update:** Updates the timer that blocks rapid crouch toggling.

Public API

- **Set Control Mode:** Change between automatic or manual input control.
- **Set Player Ability:** Assign abilities like jump or crouch.
- **Enable/Disable Running:** Toggle whether the player can run.
- **Toggle Playable:** Enable or disable player input.
- **Save Player Data:** Serializes player state (position, rotation, abilities, etc.) for later loading.
- **Load Player Data:** Restores player state from saved data.

Cleanup

- Disposes sensors, controllers, and input events to prevent memory leaks.

Usage

1. Attach the controller to a player object.

2. Assign references for input actions, Rigidbody, colliders, and physics materials.
3. Configure movement, jump, and stamina parameters.
4. The player now responds to movement input, can jump, run, crouch, and interacts correctly with obstacles and ground surfaces.
5. Optionally, use the public API to toggle abilities, change control modes, or save/load the player state.

In short, this script is a complete framework for 3D character movement with modular sensors and controllers, allowing flexible gameplay mechanics like running, jumping, crouching, and slope detection. It ensures smooth and consistent player behavior while giving developers tools to extend or modify its functionality.